

分类：基本概念和方法

6

分类是一种数据分析形式，它提取描述重要数据类别的模型。这些模型称为分类器，能够预测分类（离散、无序）标签。例如，我们可以构建一个分类模型，将银行贷款申请分类为安全或风险，或者根据患者的功能性磁共振成像（fMRI）扫描识别认知障碍的早期迹象，或者帮助自动驾驶汽车自动识别各种交通标志。这种分析可以帮助我们更好地理解整体数据。许多分类方法已被机器学习、模式识别和统计学研究人员提出。传统的分类算法通常假设数据量较小或中等。现代分类技术在此基础上发展，开发出可扩展的分类和预测技术，能够处理非常大的数据量。分类属于监督学习，并与许多其他数据挖掘任务密切相关。分类有许多应用，包括欺诈检测、目标市场营销、性能预测、制造、医学诊断等。

我们首先在第 6.1 节介绍分类的主要思想。在本章的其余部分，您将学习数据分类的基本技术，例如如何构建决策树分类器（第 6.2 节）、贝叶斯分类器（第 6.3 节）、懒惰学习者（第 6.4 节）和线性分类器（第 6.5 节）。第 6.6 节讨论如何评估和比较不同的分类器。提供了各种准确度的度量，以及获取可靠准确度估计的技术。第 6.7 节介绍了提高分类器准确度的方法，包括集成方法和类别不平衡数据（即，主要关注的类别很少）。

6.1 基本概念

我们在第 6.1.1 节介绍分类的概念。第 6.1.2 节描述了将分类作为一个两步过程的一般方法。在第一步中，我们基于以前的数据构建分类模型。在第二步中，我们确定模型的准确性是否可接受，如果可以，我们使用该模型对新数据进行分类。

6.1.1 什么是分类？

银行贷款官需要分析她的数据，以了解哪些贷款申请人对银行是“安全的”，而哪些是“风险的”，而她的风险管理部门的同事希望检测欺诈交易。电子商店的市场经理需要数据分析来帮助猜测具有特定档案的客户是否会购买新电脑，或者理解社交媒体帖子对新发布产品的情感，或者检测关于新产品的虚假评论。

在线评论网站，或识别可能转向竞争电子商店的订阅客户（即流失预测）。IT安全分析师想知道网络系统是否受到攻击（入侵检测），或者某个应用程序是否被恶意软件感染（恶意软件检测）。

一位教师希望知道一名注册在线课程的学生是否会在完成课程之前退课。一位人才招聘者想知道某个人是否在寻找下一个职业机会。一位医学研究人员希望分析乳腺癌数据，以预测患者应该接受哪三种特定治疗中的一种；一位心脏病学家希望根据患者的慢性病史识别出可能发生充血性心力衰竭的患者；一位神经科学家希望根据患者的功能性磁共振成像（fMRI）扫描识别出认知障碍的早期迹象（这可能导致，例如，阿尔茨海默病）。一个智能问答系统需要理解用户提出的问题类型（问题分类），作为自动提供高质量答案的第一步。一辆自动驾驶汽车需要自动识别各种交通标志（例如，“停车”、“绕行”等）。一位物理学家需要从大量实验数据中识别出高能事件，这可能导致新的发现。执法部门希望预测犯罪热点，以便能够主动采取预防措施。

在这些例子中，数据分析任务是分类，构建一个模型或分类器来预测类别（分类）标签，例如贷款申请数据中的“安全”或“风险”；情感分类中的“积极”或“消极”；市场数据中的“是”或“否”；在线课程注册中的“退课”或“继续”；医疗数据中的“治疗 A”、“治疗 B”或“治疗 C”；或者问答系统中的各种问题类型。这些类别可以用离散值表示，其中值之间的顺序没有意义。例如，值 1、2 和 3 可以用来表示治疗 A、B 和 C，而在这一组治疗方案中并没有暗示顺序。

假设市场经理想预测某个客户在促销期间的消费金额；或者房地产经纪人可能对不同住宅区下一年的平均房价感兴趣；或者职业规划师想预测不同专业的学生在大学毕业后立即的平均年收入。这种数据分析任务是数值预测的一个例子，其中构建的模型预测一个连续值函数，或有序值，而不是类别标签。回归分析是一种最常用于数值预测的统计方法；因此这两个术语往往可以互换使用，尽管还有其他数值预测的方法。排名是另一种数值预测类型，其中模型预测有序值（即排名），例如，网络搜索引擎（如 Google）根据给定查询对相关网页进行排名，排名越高的网页与查询的相关性越强。

分类和数值预测是两种主要的预测问题。本章主要集中于分类。值得指出的是，分类和数值预测（例如，回归）彼此密切相关。许多分类技术可以为回归的目的进行修改。我们将看到一些例子，包括回归树（第6.2节）、懒惰学习者（第6.4.1节）、线性回归（第6.5节）和梯度树提升（第6.7.1节）。

6.1.2 分类的一般方法

“分类是如何工作的？”数据分类是一个两步过程，包括学习步骤（构建分类模型的步骤）和分类步骤（使用模型的步骤）

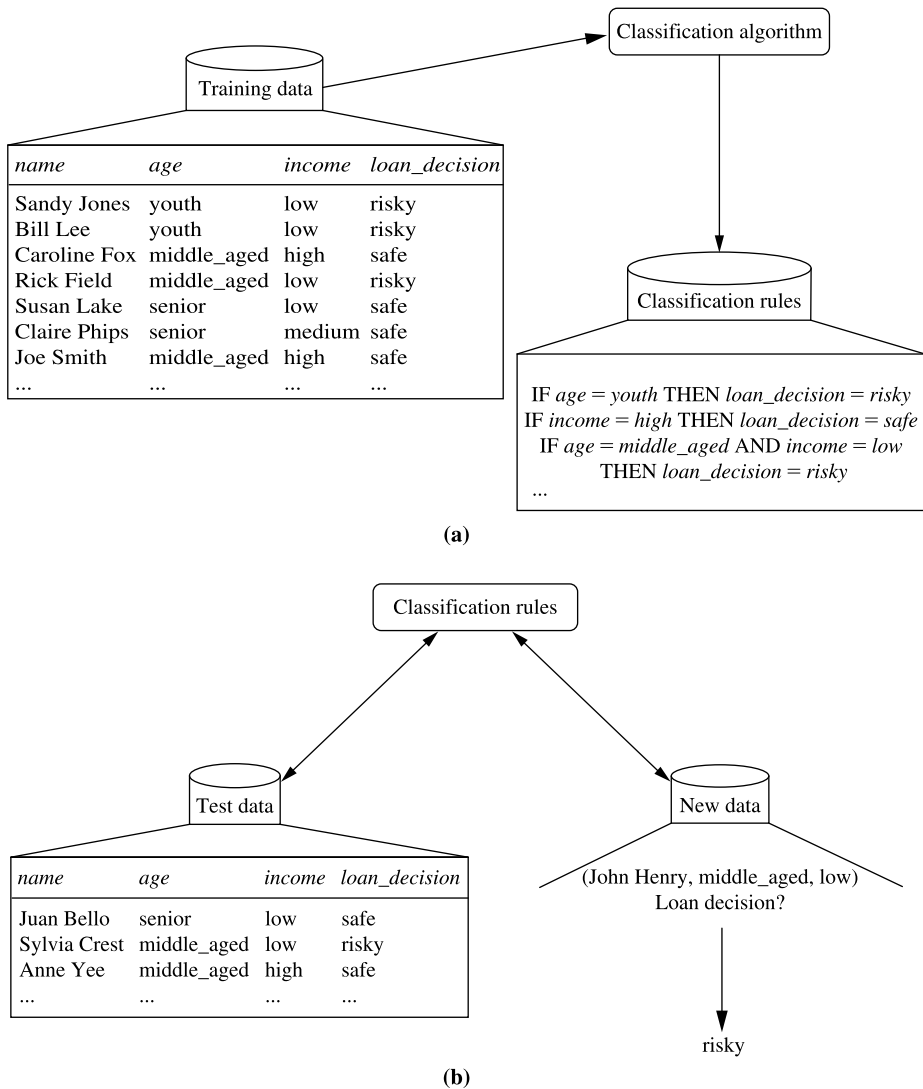


图 6.1

数据分类过程：(a) 学习：训练数据通过分类算法进行分析。在这里，类标签属性是 *loan_decision*，学习到的模型或分类器以分类规则的形式表示。

(b) 分类：测试数据用于估计分类规则的准确性。如果准确性可以接受，这些规则可以应用于新数据元组的分类。

为给定数据预测类别标签的过程如图 6.1 所示，针对贷款申请数据。数据经过简化以便于说明。实际上，我们可能会期望考虑更多的属性。

在第一步中，构建一个分类器，描述一组预定的数据类别或概念。这是学习步骤（也称为训练阶段），在此阶段，分类算法通过分析或“从”训练集学习来构建分类器，训练集由数据库元组及其相关的类别标签组成。一个元组， X ，表示为一个 n 维属性向量， $X = (x_1, x_2, \dots, x_n)$ ，描绘了从 n 个数据库属性 $A_1, A_2, \dots, A_n$ 上对元组进行的 n 个测量。每个元组 X 被假定属于由另一个数据库属性称为类别标签属性所确定的预定义类别。类别标签属性是离散值且无序的。它是分类的（或名义的），因为每个值作为一个类别或类。构成训练集的单个元组称为训练元组，并且是从正在分析的数据库中随机抽样的。在分类的上下文中，数据元组可以称为样本、示例、实例、数据点或对象。

因为每个训练元组的类标签是已提供的，因此这一步属于监督学习（即，分类器的学习是“监督的”，因为它被告知每个训练元组属于哪个类）。监督学习的范围大于分类，广泛涵盖用于训练数值预测模型的学习方法（例如，回归、排序），如果在学习步骤中已知训练元组的真实目标值。监督学习与无监督学习（例如，聚类）形成对比，在无监督学习中，每个训练元组的真实目标值（例如，类标签）是未知的，并且要学习的类的数量或集合可能事先未知。例如，如果我们没有 *loan_decision* 数据可用于训练集，我们可以使用聚类来尝试确定“相似元组的组”，这些组可能对应于贷款申请数据中的风险组。同样，我们可以使用聚类技术来找到分享相似主题的社交媒体帖子，而不知道它们的实际类标签。聚类是第8章和第9章的主题。预测问题的格局（例如，分类、回归、排序）已经超越了监督学习与无监督学习的对比。举几个例子，在半监督分类中，它基于有限数量的标记训练元组（在训练期间给出其真实类标签）和大量未标记训练元组（在训练期间其类标签未知）构建分类器；在零样本学习中，某些类标签可能在分类模型构建后出现。换句话说，在训练阶段，对于这样的类标签，没有（即，零）标记训练元组。

半监督学习和零样本学习都属于弱监督学习，因为用于训练模型的监督信息比标准监督学习要弱。对于分类任务，这意味着在半监督学习中，监督（即训练元组的真实类别标签）仅在整个训练集的一小部分中是已知的；而在零样本学习中，某些类别标签是缺失的。弱监督下的分类将在第7章中介绍。

分类过程的第一步也可以视为学习一个映射或函数， $y = f(X)$ ，该函数可以预测给定元组 X 的相关类别标签 y 。从这个角度来看，我们希望学习一个能够区分数据类别的映射或函数。通常，这个映射以分类规则、决策树或数学公式的形式表示。在我们的例子中，映射-

¹ 每个属性代表 X 的一个“特征”。因此，模式识别文献使用术语 *特征向量* 而不是 *属性向量*。在我们的讨论中，我们将这两个术语交替使用。在我们的符号中，任何表示向量的变量通常以粗斜体字显示；表示向量的测量值以斜体字显示（例如， $X = (x_1, x_2, x_3)$ ）。

² 在机器学习文献中，训练元组通常被称为 *训练样本*。本文中，我们更倾向于使用术语 *元组* 而不是 *样本*。

ping被表示为分类规则，这些规则将贷款申请识别为安全或风险

（图6.1（a））。这些规则可以用于对未来的数据元组进行分类，并提供更深入的数据内容洞察。它们还提供了一种压缩的数据表示。

“分类准确率如何？”在第二步（图6.1(b)）中，模型用于分类。首先，估计分类器的预测准确率。如果我们使用训练集来衡量分类器的准确性，这个估计可能会过于乐观，因为分类器往往会过拟合数据（即，在学习过程中，它可能会包含一些训练数据的特定异常，这些异常并不代表一般数据集）。因此，使用一个测试集，包含测试元组及其相关的类别标签。它们与训练元组是独立的，这意味着它们没有被用于构建分类器。

分类器在给定测试集上的准确性是正确分类的测试元组所占的百分比。每个测试元组的相关类标签与学习到的分类器对该元组的类预测进行比较。第6.6节描述了几种估计分类器准确性的方法。如果分类器的准确性被认为是可接受的，则可以使用该分类器对未来类标签未知的数据元组进行分类。这类数据在机器学习文献中也被称为“未知”或“之前未见过”的数据。例如，从之前贷款申请的数据分析中学习到的分类规则（见图6.1（a））可以用于批准或拒绝新的或未来的贷款申请人。

6.2 决策树生成

决策树归纳是从带类标签的训练元组中学习决策树。决策树是一种类似流程图的树结构，其中每个内部节点（非叶节点）表示对一个属性的测试，每个分支代表测试的结果，每个叶节点（或终端节点）持有一个类标签。树中的最上层节点是根节点。图6.2显示了一个典型的决策树。它表示概念 `buys_computer`；即，它预测在电子商店的顾客是否会购买计算机。

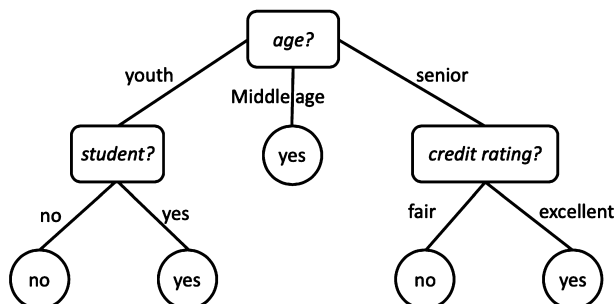


图 6.2

一个关于概念 `buys_computer` 的决策树，指示客户是否可能购买计算机。每个内部（非叶子）节点表示对一个属性的测试。每个叶子节点表示一个类别（即 `buys_computer = 是` 或 `buys_computer = 否`）。

可能购买计算机。内部节点用矩形表示，叶子节点用椭圆（或圆形）表示。一些决策树算法只生成二叉树（每个内部节点分支到恰好两个其他节点），而其他算法可以生成非二叉树。

“决策树如何用于分类？”给定一个元组 X ，其关联的类别标签未知，元组的属性值会与决策树进行测试。从根节点到叶子节点追踪一条路径，该叶子节点保存该元组的类别预测。决策树可以很容易地转换为分类规则。

“为什么决策树分类器如此受欢迎？”构建决策树分类器不需要任何领域知识或参数设置，因此适合用于探索性知识发现。决策树可以处理多维数据。它们以树形表示所获得的知识直观且通常易于人类理解。决策树归纳的学习和分类步骤简单且快速。一般来说，决策树分类器具有良好的准确性。然而，成功的使用可能依赖于手头的数据。决策树归纳算法已在医学、制造与生产、金融分析、天文学和分子生物学等多个应用领域用于分类。决策树是多个商业规则归纳系统的基础。

在第6.2.1节中，我们描述了一种学习决策树的基本算法。在树的构建过程中，属性选择度量被用来选择最佳划分元组为不同类别的属性。常用的属性选择度量在第6.2.2节中给出。当决策树被构建时，许多分支可能反映了训练数据中的噪声或异常值。树剪枝试图识别并移除这些分支，目的是提高对未见数据的分类准确性。

树木修剪在第6.2.3节中描述。

6.2.1 决策树生成

在1970年代末和1980年代初，机器学习研究员J. Ross Quinlan开发了一种称为ID3（迭代二分法）的决策树算法。这项工作扩展了E. B. Hunt、J. Marin和P. T. Stone所描述的早期概念学习系统的研究。Quinlan随后提出

C4.5（ID3的继任者），成为了许多新监督学习算法常常比较的基准。1984年，一组统计学家（L. Breiman、J. Friedman、R. Olshen和C. Stone）出版了书籍《分类与回归树》（CART），描述了二元决策树的生成。ID3和CART大约在同一时间独立发明，但在从训练元组学习决策树的方式上遵循了类似的方法。这两种基石算法催生了大量关于决策树归纳的研究。

ID3、C4.5和CART采用贪心（即非回溯）的方法，以自上而下的递归分治方式构建决策树。大多数决策树归纳算法也遵循自上而下的方法，从一组训练元组及其相关的类别标签开始。随着树的构建，训练集被递归地划分为更小的子集。基本的决策树算法在图6.3中进行了总结。乍一看，算法可能显得冗长，但不用担心！它其实相当简单。策略如下。

- 该算法调用时有三个参数：D、attribute_list和Attribute_selection_method。D是数据分区。最初，它是完整的训练元组及其相关类别标签的集合。
参数attribute_list是描述元组的属性列表。Attribute_selection_method指定了一种启发式程序，用于选择“最佳”区分给定元组的属性。

算法：Generate_decision_tree。从数据分区 D 的训练元组生成决策树。

输入：

- 数据分区 D ，它是一组训练元组及其相关类别标签；
- 属性列表，候选属性的集合；
- 属性选择方法，是一种确定“最佳”划分标准的程序，该标准将数据元组划分为各个类别。该标准由一个划分属性和可能的一个划分子集组成。

输出：一棵决策树。

方法：

- (1) 创建一个节点 N ；
- (2) 如果 D 中的元组都是同一类别 C ，则
- (3) 返回 N 作为标记为类别 C 的叶节点；
- (4) 如果 属性列表 为空 则
- (5) 返回 N 作为一个标记为 D 中多数类的叶节点； // 多数投票
- (6) 应用 属性选择方法(D , 属性列表)来 找到 “最佳” 划分标准；
- (7) 用 划分标准 标记节点 N ；
- (8) 如果 划分属性 是离散值 并且
 允许多路划分 则 // 不限于二叉树
- (9) 属性列表 $\leftarrow$ 属性列表 - 划分属性； // 移除 划分属性
- (10) 对于每个 划分标准的结果 j
 // 对元组进行分区并为每个分区生长子树
- (11) 让 D_j 为满足结果 j 的数据元组集合 D ； // 一个划分
- (12) 如果 D_j 为空 则
- (13) 将标记为 D 中多数类的叶子附加到节点 N ；
- (14) 否则 将Generate_decision_tree(D_j , attribute_list) 返回的节点附加到节点 N ；
- 结束
- (15) 返回 N 。

图 6.3

从训练元组诱导决策树的基本算法。

根据类别。该过程采用属性选择度量，例如信息增益或基尼不纯度。（我们将在下一小节中介绍这些度量。）树是否严格为二叉树通常由属性选择度量驱动。一些属性选择度量，例如基尼不纯度，强制生成的树为二叉树。其他度量，如信息增益，则不强制，从而允许多路分裂（即从一个节点生长出两个或更多分支）。

- 树开始时是一个单节点， N ，表示训练元组在 D 中（步骤 1）。³
- 如果 D 中的元组都属于同一类别，则节点 N 变为叶子节点，并标记为该类别（步骤 2 和 3）。请注意，步骤 4 和 5 是终止条件。所有终止条件在算法的最后进行了说明。
- 否则，算法调用Attribute_selection_method 来确定 切分标准。切分标准告诉我们在节点 N 测试哪个属性，通过确定“最佳”的分离方式

³ 在节点 N 处的类标签训练元组的划分是从树的根节点到节点的路径上遵循的一组元组 N 在树中处理时。这个集合在文献中有时被称为 节点 N 的元组族。我们称这个集合为“在节点 N 处表示的元组”、“到达节点 N 的元组”，或简单地称为“在节点 N 的元组。”大多数实现并不存储节点处的实际元组，而是存储指向这些元组的指针。

或将数据集 D 中的元组划分为单独的类别（步骤 6）。分割标准还告诉我们根据所选测试的结果，从节点 N 应该生长哪些分支。更具体地说，分割标准指示分割属性，并可能还指示分割点或分割子集。分割标准的确定旨在使每个分支的结果分区尽可能“纯”。如果分区中的所有元组都属于同一类别，则该分区是纯的。

换句话说，如果我们根据分裂标准的互斥结果将元组在 D 中拆分，我们希望得到的分区尽可能纯净。

- 节点 N 被标记为分裂标准，作为节点的测试（步骤 7）。从节点 N 为分裂标准的每个结果生长一条分支。元组在 D 中相应地被划分（步骤 10–11）。如图 6.4 所示，有三种可能的情况。

设 A 为分裂属性。 A 具有 v 个不同的值， $\{a_1, a_2, \dots, a_v\}$ ，基于训练数据。

1. A 是离散值：在这种情况下，节点 N 的测试结果直接对应于 A 的已知值。为 A 的每个已知值 a_j 创建一条分支，并用该值标记（图 6.4(a)）。分区 D_j 是在 D 中具有 A 的值 a_j 的类标记元组的子集。因为给定分区中的所有元组在 A 上具有相同的值，所以 A 不需要被

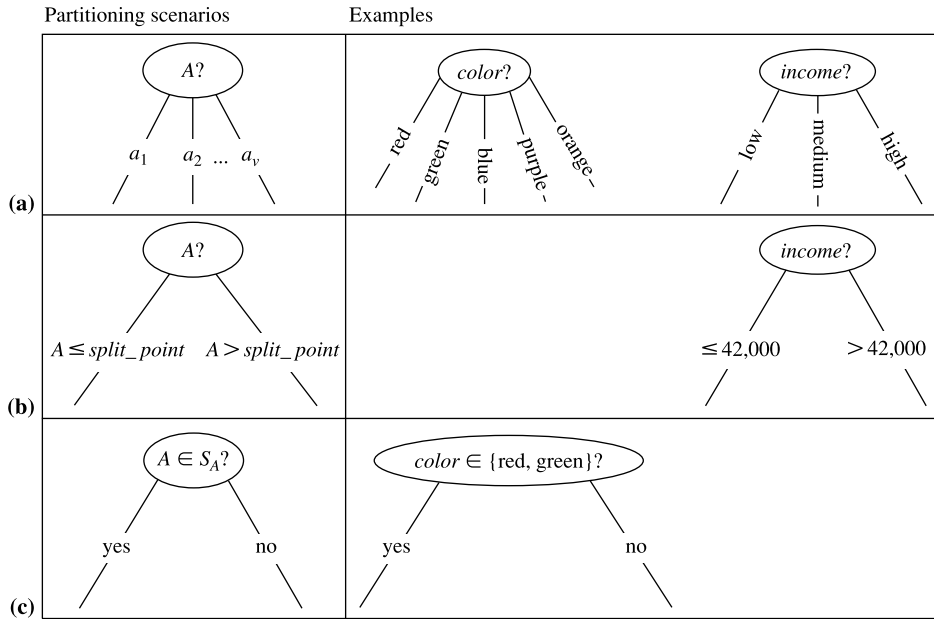


图 6.4

该图显示了基于分裂标准对元组进行划分的三种可能性，每种情况都有示例。设 A 为分裂属性。(a) 如果 A 是离散值，则为 A 的每个已知值生长一个分支。(b) 如果 A 是连续值，则生长两个分支，分别对应于 $A \leq \text{split_point}$ 和 $A > \text{split_point}$ 。(c) 如果 A 是离散值且必须生成二叉树，则测试形式为 $A \in S_A$ ，其中 S_A 是 A 的分裂子集。

在未来的元组划分中考虑。因此，它从`attribute_list`

(步骤 8 和 9)中移除。

2. *A 是连续值*：在这种情况下，节点 *N* 的测试有两个可能的结果，分别对应于条件 $A \leq \text{split_point}$ 和 $A > \text{split_point}$ ，其中 `split_point` 是由 `Attribute_selection_method` 返回的分裂点，作为分裂标准的一部分。(在实践中，分裂点 *a* 通常取为 *A* 的两个已知相邻值的中点，因此可能并不是训练数据中 *A* 的一个预先存在的值。)从 *N* 生长两个分支，并根据先前的结果进行标记 (图 6.4(b))。元组被划分为 *D*₁，包含在 *D* 中 $A \leq \text{split_point}$ 的类标记元组的子集，而 *D*₂ 则包含其余部分。
3. *A 是离散值的，必须生成一个二叉树* (由所使用的属性选择度量或算法决定)：节点 *N* 的测试形式为 “ $A \in S_A?$ ”，其中 *S_A* 是 *A* 的分割子集，由 `Attribute_selection_method` 作为分割标准的一部分返回。它是已知值的 *A* 的一个子集。如果给定元组的 *A* 的值为 *a_j*，且 $a_j \in S_A$ ，则节点 *N* 的测试满足。两个分支从 *N* 生长 (图 6.4 (c))。根据约定，从 *N* 的左分支标记为 *yes*，以便 *D*₁ 对应于在 *D* 中满足测试的类标记元组的子集。从 *N* 的右分支标记为 *no*，以便 *D*₂ 对应于在 *D* 中不满足测试的类标记元组的子集。

- 该算法使用相同的过程递归地为每个结果分区 *D_j* 形成决策树，*D* (步骤 14)。
- 递归分区仅在以下终止条件之一为真时停止：
 1. 分区 *D* (在节点 *N* 表示) 中的所有元组属于同一类 (步骤 2 和 3)。
 2. 没有剩余的属性可以进一步对元组进行划分 (步骤 4)。在这种情况下，采用多数投票 (步骤 5)。这涉及将节点 *N* 转换为叶子，并用 *D* 中最常见的类别进行标记。或者，可以存储节点元组的类别分布。
 3. 对于给定的分支，没有元组，即划分 *D_j* 是空的 (步骤 12)。在这种情况下，创建一个叶子，标记为 *D* 中的多数类 (步骤 13)。
- 返回生成的决策树 (步骤 15)。

给定训练集 *D* 的算法计算复杂度为 $O(n \times |D| \times \log(|D|))$ ，其中 *n* 是描述 *D* 中元组的属性数量，*|D|* 是 *D* 中训练元组的数量。这意味着，生成一棵树的计算成本最多为 $n \times |D| \times \log(|D|)$ ，与 *|D|* 元组相关。证明留给读者作为练习。

还提出了增量版本的决策树诱导。当给定新的训练数据时，它会重构从先前训练数据学习获得的决策树，而不是从头开始重新学习一棵新树。

决策树算法的差异包括在创建树时如何选择属性 (第 6.2.2 节) 以及用于剪枝的机制 (第 6.2.3 节)。

决策树与另一种树类型密切相关，称为回归树，用于预测连续输出值。回归树与决策树非常相似，因为它也将整个属性空间划分为多个子区域，每个子区域对应一个叶节点。主要区别如下。在回归树中，叶节点持有一个连续值，而不是一个

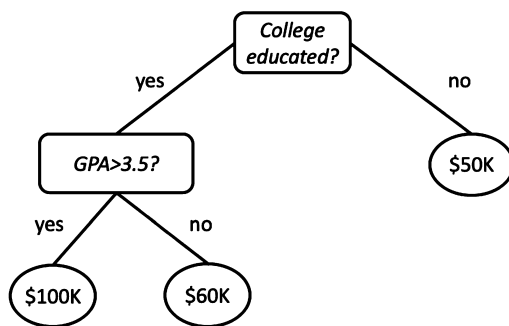


图 6.5

一个回归树，用于根据个人的教育水平预测平均年收入。三个叶节点的值计算如下。 $\$50K$ 是所有没有大学学位的培训个体的平均年收入； $\$60K$ 是所有拥有大学学位且 GPA 小于或等于 3.5 的培训个体的平均年收入； $\$100K$ 是所有拥有大学学位且 GPA 高于 3.5 的培训个体的平均年收入。叶节点值（ $\$50K$ 、 $\$60K$ 和 $\$100K$ ）用于预测任何落入相应叶节点的测试个体的年收入。

决策树中的分类值（即类标签）。叶节点的连续值是在训练阶段学习得出的，该值被设定为所有落入相应子区域的训练元组的平均输出值。CART 使用残差平方和（RSS）作为目标函数，它是训练元组的实际输出值与预测输出值之间平方差的总和。

$$RSS = \sum_i (y_i - \hat{y}_i)^2, \quad (6.1)$$

其中 y_i 是第 i 个训练元组的实际输出值， $\hat{y}_i$ 是回归树预测的输出。选择相应子区域中所有训练元组的平均输出是最优的，因为它最小化了公式 (6.1) 中的 RSS。每个叶节点值随后用于预测落入该节点的测试元组的输出。图 6.5 展示了一个回归树的示例，用于根据个人的教育背景（例如，个人是否上过大学、大学的平均 GPA 等）预测平均年收入。

6.2.2 属性选择度量

属性选择度量是一种启发式方法，用于选择“最佳”分割标准，以将类标签训练元组的数据分区 D 分离成各个类。如果我们根据分割标准的结果将 D 分割成更小的分区，理想情况下，每个分区都是纯的（即，落入给定分区的所有元组都属于同一类）。从概念上讲，“最佳”分割标准是最接近实现这种情况的标准。属性选择度量也被称为分割规则，因其决定了在给定的节点处如何分割元组。

属性选择度量为描述给定训练样本的每个属性提供了一个排名。得分最佳的属性被选择为给定样本的分裂属性。如果分裂属性是连续值，或者我们限制为二叉树，则必须分别确定一个分裂点或一个分裂子集作为分裂标准的一部分。为划分的 D 创建的树节点标记为分裂标准，针对标准的每个结果生长分支，并相应地对样本进行划分。本节描述了三种流行的属性选择度量——信息增益、增益比和基尼不纯度。

本文中使用的符号如下。设 D 为数据分区，是一个带有类标签的训练集 n 元组。假设类标签属性有 m 个不同的值，定义 m 个不同的类， C_i （对于 $i=1, \dots, m$ ）。设 $C_{i,D}$ 为在 D 中类 C_i 的元组集合。设 $|D|$ 和 $|C_{i,D}|$ 分别表示 D 和 $C_{i,D}$ 中元组的数量。

信息增益

ID3 使用信息增益作为其属性选择度量。该度量基于克劳德·香农在信息理论方面的开创性工作，研究了消息的价值或“信息内容”。让节点 N 表示或包含分区 D 的元组。具有最高信息增益的属性被选择为节点 N 的分裂属性。该属性最小化了对结果分区中元组进行分类所需的信息，并反映了这些分区中最少的随机性或“杂质”。这种方法最小化了对给定元组进行分类所需的预期测试次数，并保证找到一个简单的（但不一定是最简单的）树。

分类元组所需的预期信息在 D 中给出

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i), \quad (6.2)$$

其中 p_i 是一个任意元组在 D 中属于类 C_i 的非零概率，估计为 $|C_{i,D}|/|D|$ 。使用以 2 为底的对数函数，因为信息以比特编码。 $Info(D)$ 是识别 D 中元组的类标签所需的平均信息量。请注意，此时我们拥有的信息仅基于每个类的元组比例。 $Info(D)$ 也被称为 D 的熵。

现在，假设我们要根据某个属性 A 将 D 中的元组划分为具有 v 个不同值的子集， $\{a_1, a_2, \dots, a_v\}$ ，这是从训练数据中观察到的。如果 A 是离散值，这些值直接对应于对 A 的测试的 v 个结果。属性 A 可以用来将 D 划分为 v 个分区或子集， $\{D_1, D_2, \dots, D_v\}$ ，其中 D_j 包含在 D 中具有 A 的结果 a_j 的元组。这些分区将对应于从节点 N 生长出的分支。理想情况下，我们希望这种划分能够产生元组的精确分类。也就是说，我们希望每个分区都是纯净的。

然而，分区很可能是杂质的（例如，一个分区可能包含来自不同类别的元组集合，而不是来自单一类别）。

⁴ 根据度量的不同，选择最高或最低分数作为最佳（即，有些度量旨在最大化，而其他度量则旨在最小化）。

在进行分区后，我们还需要多少更多的信息才能得出准确的分类？这个数量由

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j). \quad (6.3)$$

术语 $|D_j|$ 作为第 j 个分区的权重。 $Info_A(D)$ 是根据 A 的分区来分类来自 D 的元组所需的期望信息。所需的期望信息越小，分区的纯度越高。 $Info_A(D)$ 也被称为 D 的条件熵（以属性 A 为条件）。

信息增益被定义为原始信息需求（即，仅基于类别的比例）与新需求（即，在 A 上进行分区后获得的）之间的差异。也就是说，

$$Gain(A) = Info(D) - Info_A(D). \quad (6.4)$$

换句话说， $Gain(A)$ 告诉我们通过对 A 进行分支可以获得多少收益。它是由于知道 A 的值而导致的信息需求的预期减少。 A 的信息增益最高， $Gain(A)$ ，被选择为节点 N 的分裂属性。这等同于说我们希望在属性 A 上进行划分，以实现“最佳分类”，从而使完成分类元组所需的信息量最小（即，最小 $Info_A(D)$ ）。

示例 6.1. 使用信息增益诱导决策树。表 6.1 展示了一个训练集， D ，包含从电子商店的客户数据库中随机选择的带类标签的元组。（这些数据改编自 Quinlan [Qui86]。在这个例子中，每个属性都是离散值。连续值属性已被概括。）类标签属性，*buys_computer*，有两个不同的

表 6.1 来自电子商店客户数据库的带类标签的训练元组。

RID年龄		收入学生信用评级类: buys_computer			
1	青年	高	否	一般	否
2	青年	高	否	优秀	否
3	中年	高	否	一般	是
4	高级	中等	否	一般	是
5	高级	低	是	一般	是
6	高级	低	是	优秀	否
7	中年	低	是	优秀	是
8	青年	中等	否	一般	否
9	青年	低	是	一般	是
10	高级	中等	是	一般	是
11	青年	中等	是	优秀	是
12	中年	中等	否	优秀	是
13	中年	高	是	一般	是
14	高级	中等	否	优秀	否

值（即， $\{yes, no\}$ ）；因此，有两个不同的类别（即， $m=2$ ）。让类别 C_1 对应于 yes ，类别 C_2 对应于 no 。类别 yes 有九个元组，类别 no 有五个元组。为数据集 D 中的元组创建一个（根）节点 N 。为了找到这些元组的分裂标准，我们必须计算每个属性的信息增益。我们首先使用公式（6.2）计算分类元组所需的期望信息：

$$Info(D) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) = 0.940 \text{ bits.}$$

接下来，我们需要计算每个属性的期望信息需求。我们从属性 age 开始。我们需要查看每个年龄类别的 yes 和 no 元组的分布。对于年龄类别“青年”，有两个 yes 元组和三个 no 元组。对于类别“中年”，有四个 yes 元组和零个 no 元组。对于类别“老年”，有三个 yes 元组和两个 no 元组。使用公式（6.3），如果元组根据 age 进行划分，则分类元组所需的期望信息为

$$\begin{aligned} Info_{age}(D) &= \frac{5}{14} \times \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) \\ &\quad + \frac{4}{14} \times \left(-\frac{4}{4} \log_2 \frac{4}{4} \right) \\ &\quad + \frac{5}{14} \times \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right) \\ &= 0.694 \text{ bits.} \end{aligned}$$

因此，从这种划分中获得的信息增益将是

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

同样，我们可以计算 $Gain(income)=0.029$ bits, $Gain(student)=0.151$ bits, 以及 $Gain(credit_rating)=0.048$ bits。因为 age 在所有属性中具有最高的信息增益，它被选为分裂属性。节点 N 被标记为 age ，且为每个属性值生长分支。然后，元组根据属性值进行相应的划分，如图6.6所示。注意，落入 $age=middle_aged$ 划分的元组都属于同一类别。因为它们都属于类别“yes，”因此在此分支的末尾应创建一个叶子，并标记为“yes。”算法返回的最终决策树在图6.2中已显示。

□

“但是我们如何计算一个连续值属性的信息增益呢，这与示例中的情况不同？”假设我们有一个属性 A ，它是连续值而不是离散值。（例如，假设我们有这个属性的原始值，而不是示例中的离散版本的 age 。）在这种情况下，我们必须确定 A 的“最佳”切分点，切分点是 A 的一个阈值。

我们首先将 A 的值按升序排序。通常，考虑每对相邻值之间的中点作为可能的切分点。因此，给定 v 个 A 的值，评估 $(v-1)$ 个可能的切分。例如， A 的值 a_i 和 a_{i+1} 之间的中点是

$$\frac{a_i + a_{i+1}}{2}. \quad (6.5)$$

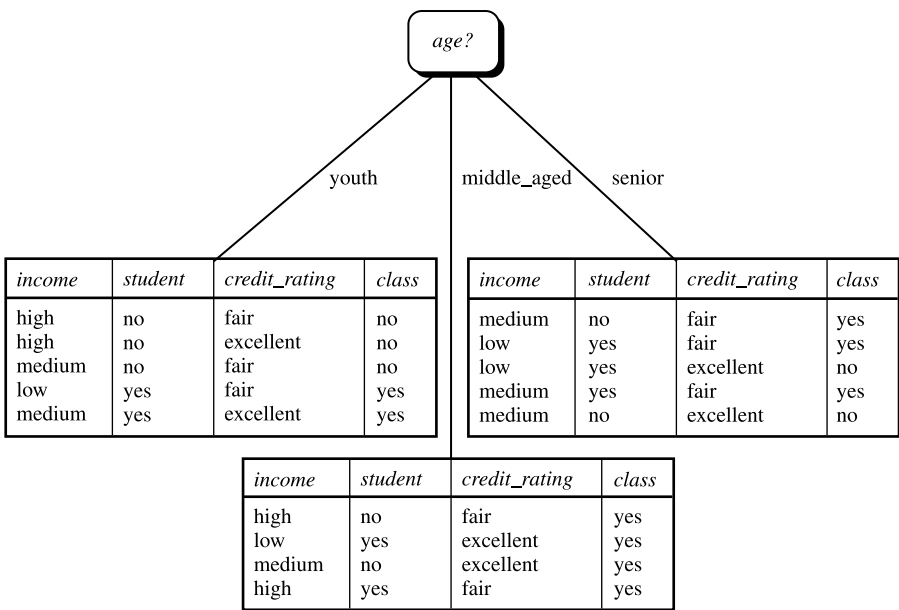


图 6.6

属性 age 具有最高的信息增益，因此成为决策树根节点的切分属性。为 age 的每个结果生长分支。元组按此方式进行分区。

如果 A 的值事先已排序，那么确定 A 的最佳切分只需对值进行一次遍历。对于 A 的每个可能切分点，我们评估 $Info_A(D)$ ，其中分区数量为二，即 $v=2$ （或 $j=1, 2$ ）在公式 (6.3) 中。选择 A 的最小预期信息需求的点作为 A 的 $split_point$ 。 D_1 是满足 $A \leq split_point$ 的元组集合， D_2 是满足 $A > split_point$ 的元组集合。

增益比

信息增益度量偏向于具有多个结果的测试。也就是说，它更倾向于选择具有大量值的属性。例如，考虑一个作为唯一标识符的属性，如 $product_ID$ 。对 $product_ID$ 的划分将导致大量的分区（与值的数量相同），每个分区仅包含一个元组。由于每个分区都是纯的，因此基于这种划分对数据集 D 进行分类所需的信息将是 $Info_{product_ID}(D)=0$ 。因此，通过对该属性进行划分所获得的信息是最大的。显然，这种划分对分类是无用的。

C4.5是ID3的继任者，使用了一种称为 **增益比** 的信息增益扩展，试图克服这种偏见。它对信息增益应用了一种归一化，使用与 $Info(D)$ 类似定义的“划分信息”值

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right). \tag{6.6}$$

该值表示通过将训练数据集 D 划分为 v 个分区（对应于属性 A 上测试的 v 个结果）所生成的潜在信息。请注意，对于每个结果，它考虑了具有该结果的元组数量与 D 中元组总数的比例。

它与信息增益不同，信息增益是基于相同划分所获得的与分类相关的信息度量。增益比定义为

$$\text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}_A(D)}. \quad (6.7)$$

具有最大增益比的属性被选为分裂属性。然而，请注意，当分裂信息接近0时，比例变得不稳定。为避免这种情况，增加了一个约束，即所选测试的信息增益必须很大——至少与所有检查过的测试的平均增益相等。

示例6.2. 属性收入 的增益比计算。对收入的测试将表6.1的数据分成三个部分，即低、中和高，分别包含四个、六个和四个元组。为了计算收入的增益比，我们首先使用公式(6.6)得到

$$\begin{aligned} \text{SplitInfo}_{\text{income}}(D) &= -\frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) - \frac{6}{14} \times \log_2 \left(\frac{6}{14} \right) - \frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) \\ &= 1.557. \end{aligned}$$

从示例6.1中，我们有 $\text{Gain}(\text{收入}) = 0.029$ 。因此 $\text{GainRatio}(\text{收入}) = 0.029/1.557 = 0.019$ 。

□

基尼不纯度

基尼不纯度（简称基尼）在CART中使用。使用之前描述的符号，基尼度量数据划分或训练元组集 D 的不纯度，如下所示

$$\text{Gini}(D) = 1 - \sum_{i=1}^m p_i^2, \quad (6.8)$$

其中 p_i 是元组在 D 中属于类 $C \setminus i$ 的概率，估计为 $|C \setminus i, D|/|D|$ 。该和在 m 类上计算。

基尼不纯度考虑了每个属性的二元分裂。我们首先考虑属性 A 是一个离散值属性，具有 v 个不同的值， $\{a_1, a_2, \dots, a_v\}$ ，出现在 D 中。为了确定 A 上的最佳二元分裂，我们检查所有可能的子集，这些子集可以使用已知的 A 值形成。每个子集 S_A 可以被视为属性 A 的二元测试，形式为“ $A \in S_A$ ？”给定一个元组，如果该元组的 A 值在 S_A 中列出的值之中，则该测试成立。

如果 A 有 v 个可能值，则有 2^v 个可能的子集。例如，如果 income 有三个可能值，即 $\{\text{low}, \text{medium}, \text{high}\}$ ，那么可能的子集为 $\{\text{low}, \text{medium}, \text{high}\}$ 、 $\{\text{low}, \text{medium}\}$ 、 $\{\text{low}, \text{high}\}$ 、 $\{\text{medium}, \text{high}\}$ 、 $\{\text{low}\}$ 、 $\{\text{medium}\}$ 、 $\{\text{high}\}$ 和 $\{\}$ 。我们排除幂集 $\{\text{low}, \text{medium}, \text{high}\}$ 和空集，因为从概念上讲，它们并不代表分裂。因此，基于 A 的二元分裂，形成数据 D 的两个分区的可能方式为 $(2^v - 2)/2$ 。

在考虑二元分裂时，我们计算每个结果分区的不纯度的加权和。例如，如果对 A 的二元分裂将 D 分为 D_1 和 D_2 ，则给定该分区的 D 的基尼不纯度为

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2). \quad (6.9)$$

对于每个属性，考虑所有可能的二元分割。对于离散值属性，选择使该属性的基尼不纯度最小的子集作为其分割子集。

对于连续值属性，必须考虑每个可能的分割点。该策略与之前描述的信息增益相似，其中每对（排序后的）相邻值之间的中点被视为可能的分割点。对于给定的（连续值）属性，使基尼不纯度最小的点被视为该属性的分割点。回想一下，对于属性 A 的可能分割点， D_1 是满足 $A \leq \text{split_point}$ 的 D 中的元组集合，而 D_2 是满足 $A > \text{split_point}$ 的 D 中的元组集合。

对离散或连续值属性 A 进行二元分割所带来的不纯度减少为

$$\Delta Gini(A) = Gini(D) - Gini_A(D). \quad (6.10)$$

最大化不纯度减少（或等效地，具有最小基尼不纯度）的属性被选为分割属性。该属性及其分割子集（对于离散值分割属性）或分割点（对于连续值分割属性）共同构成分割标准。

示例 6.3. 使用基尼不纯度诱导决策树。设 D 为之前在表 6.1 中显示的训练数据，其中有九个元组属于类 $buys_computer=yes$ ，剩下的五个元组属于类 $buys_computer=no$ 。为元组 D 创建一个（根）节点 N 。我们首先使用公式 (6.8) 计算 D 的基尼不纯度：

$$Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459.$$

为了找到元组 D 的分裂标准，我们需要计算每个属性的基尼不纯度。我们从属性 $income$ 开始，考虑每个可能的分裂子集。考虑子集 $\{low, medium\}$ 。这将导致在分区 D_1 中有 10 个元组满足条件 “ $income \in \{low, medium\}$ 。” 剩下的四个元组将被分配到分区 D_2 。基于此分区计算的基尼不纯度值为

$$\begin{aligned} Gini_{income \in \{low, medium\}}(D) &= \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2) \\ &= \frac{10}{14} \left(1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2\right) + \frac{4}{14} \left(1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2\right) \\ &= 0.443 \\ &= Gini_{income \in \{high\}}(D). \end{aligned}$$

同样，剩余子集的分裂基尼不纯度值为 0.458（对于子集 $\{low, high\}$ 和 $\{medium\}$ ）以及 0.450（对于子集 $\{medium, high\}$ 和 $\{low\}$ ）。因此，属性 *income* 的最佳二元分裂是在 $\{low, medium\}$ （或 $\{high\}$ ）上，因为它最小化了基尼不纯度。

评估 *年龄*，我们得到 $\{年轻, 老年\}$ （或 $\{中年\}$ ）作为 *年龄* 的最佳划分，基尼不纯度为 0.375；属性 *学生* 和 *信用评级* 均为二元，基尼不纯度值分别为 0.367 和 0.429。

因此，属性 *年龄* 和划分子集 $\{年轻, 老年\}$ 给出了最低的基尼不纯度，总体上不纯度减少为 $0.459 - 0.357 = 0.102$ 。二元划分“*年龄* $\in \{年轻, 老年\}$ ”导致了 D 中元组的不纯度最大减少，并作为划分标准返回。节点 N 被标记为该标准，从中生长出两个分支，元组被相应地划分。

□

“那么，基尼不纯度和信息增益之间有什么关系？”直观上，这两种度量都旨在量化如果我们基于给定属性划分当前节点，不纯度将减少到什么程度。信息增益根植于信息理论，基于（平均信息量的变化）来衡量识别元组类标签所需的信息不纯度。基尼不纯度与 *错误分类* 之间的关系如下。基于当前节点中的类标签分布，它告诉我们随机选择的元组被分配到随机类标签时被错误分类的可能性。基尼不纯度始终用于二元划分，而信息增益允许多路划分。在计算方面，基尼不纯度比信息增益稍微高效，因为后者涉及对数计算。然而，在实践中，这两种度量通常会导致非常相似的决策树。

其他属性选择度量

这一节关于属性选择度量的内容并不打算详尽无遗。我们展示了三种常用于构建决策树的度量。这些度量并非没有偏见。正如我们所看到的信息增益，偏向于多值属性。尽管增益比率对此偏见进行了调整，但它倾向于偏好不平衡的划分，其中一个分区远小于其他分区。基尼不纯度偏向于多值属性，并且在类别数量较多时表现困难。它还倾向于偏好导致相等大小分区和两个分区纯度的测试。

尽管存在偏见，这些度量在实践中仍能给出相当不错的结果。

许多其他属性选择度量也被提出。CHAID 是一种在市场营销中流行的决策树算法，它使用基于统计 χ^2 独立性检验的属性选择度量。其他度量包括 C-SEP（在某些情况下表现优于信息增益和基尼不纯度）和 G 统计量（一个信息理论度量，接近 χ^2 分布的近似值）。

对于回归树，使用 RSS（公式（6.1））作为划分标准是自然的。也就是说，给定属性的最佳划分点是导致最小 RSS 的点。我们选择具有最小 RSS 的属性将树节点划分为两个节点，包括左叶节点和右叶节点。

例 6.4. 让我们看一下表 6.2 中如何使用 RSS 找到最佳分割点的示例。

假设在一个回归树节点上有五个训练元组，每个训练元组都有一个真实输出值 y_i 和一个连续属性 $x_i (i=1, \dots, 5)$ 。我们想要找到属性 x_i 的最佳分割点，以将树节点分割成两个叶节点。更具体地说，所有 x_i 小于或等于分割点的元组将进入左叶节点，其余的训练元组将进入右叶节点。

表 6.2 回归的训练数据。

属性 x_i	1	2	3	4	5
输出 y_i	10	12	8	20	22

在一个回归树节点上给定五个训练元组，每个元组都有一个真实输出值 y_i 和一个连续属性 $x_i(i=1, \dots, 5)$ 。我们想要找到属性 x_i 的最佳分割点，以将树节点分割成两个节点（左节点和右节点）。

表 6.3 使用 RSS 选择表 6.2 中数据元组的最佳分割点。

候选分割点 x_i	1.5	2.5	3.5	4.5
左叶节点的预测值 y_l	10	11	10	12.5
右叶节点的预测值 y_r	15.5	16.7	21	22
RSS	131	116.67	10	83

由于 x_i 是一个具有五个可能值的连续属性，因此有四个候选分割点，包括 $x_i=1.5$, $x_i=2.5$, $x_i=3.5$ 和 $x_i=4.5$ 。对于每个候选分割点，我们将当前树节点分裂为两个叶节点。左叶节点中训练元组的平均输出值 y_l 用于预测所有位于左叶节点的元组的输出。同样，右叶节点中训练元组的平均输出值 y_r 用于预测所有位于右叶节点的元组的输出。例如，如果分裂点 $x_i=1.5$ ，则只有第一个训练元组进入左叶节点，我们有 $y_l=y_1=10$ ；而 $y_r=(y_2+y_3+y_4+y_5)/4=(12+8+$

$20+22)/4=15.5$ 。使用所有五个训练元组的预测输出值（ y_l 或 y_r ），我们可以使用 Eq. (6.1) 用于计算 RSS。再次，如果分割点 $x_i=1.5$ ，我们有 $RSS=\sum_{i=1}^5 (y_i - \hat{y}_i)^2 = (y_1 - y_l)^2 + (y_2 - y_r)^2 + (y_3 - y_r)^2 + (y_4 - y_r)^2 + (y_5 - y_r)^2 = 122.25$ 。所有四个可能的分割点的计算结果汇总在表 6.3 中。由于 $x_i=3.5$ 的 RSS 最小，因此选择它作为分割点。

□

基于最小描述长度（MDL）原则的属性选择度量对多值属性的偏见最小。基于 MDL 的度量使用编码技术来定义“最佳”决策树，即需要最少的比特数来同时（1）编码树和（2）编码树的例外情况（即未被树正确分类的案例）。其主要思想是优先选择最简单的解决方案。MDL 原则背后的哲学是

奥卡姆剃刀，也称为简约法则。在数据挖掘和机器学习中，奥卡姆剃刀通常被转化为一种设计原则，即在其他条件相同的情况下，应该偏向于描述更短的模式（因此是最小描述长度）而不是更长的模式（例如，短模型和长模型共享相同的训练集错误）。

其他属性选择度量考虑多变量分裂（即，元组的划分基于属性的组合，而不是单一属性）。例如，CART 系统可以基于属性的线性组合找到多变量分裂。多变量分裂是一种属性（或特征）构造的形式，其中基于现有属性创建新属性。（属性构造在第 2 章中也作为数据转换的一种形式进行了讨论。）这些

这里提到的其他措施超出了本书的范围。附录中提供了额外的参考文献，见本章末尾的书目注释（第 6.10 节）。

“哪种属性选择度量是最好的？”所有度量都有一定的偏差。研究表明，决策树归纳的时间复杂度通常随着树的高度呈指数增长。

因此，倾向于生成较浅树的度量（例如，使用多路分裂而非二元分裂，并且更倾向于更平衡的分裂）可能更受欢迎。然而，一些研究发现，较浅的树往往具有大量的叶子节点和更高的错误率。尽管进行了几项比较研究，但尚未发现任何单一的属性选择度量显著优于其他度量。大多数度量都能给出相当好的结果。

6.2.3 树剪枝

当构建决策树时，许多分支会反映训练数据中的异常情况，这通常是由于噪声或离群值造成的。树剪枝方法解决了数据过拟合的问题。这些方法通常使用统计度量来去除最不可靠的分支。图 6.7 显示了未剪枝的树和剪枝后的版本。剪枝后的树通常更小、更简单，因此更易于理解。与未剪枝的树相比，它们通常在正确分类独立测试数据（即以前未见过的元组）时更快且更有效。

“树剪枝是如何工作的？”树剪枝有两种常见的方法：预剪枝和后剪枝。

在预剪枝方法中，通过提前停止树的构建来“剪枝”树（例如，通过决定不再在给定节点进一步拆分或划分训练元组的子集）。一旦停止，该节点就成为叶子节点。叶子节点可以持有子集元组中最频繁类别标签或这些元组的类别标签的概率分布。

在构建树时，可以使用统计显著性、信息增益、基尼不纯度等度量来评估拆分的优劣。如果在某个节点处对元组进行划分会导致拆分低于预先指定的阈值，则停止对给定子集的进一步划分。

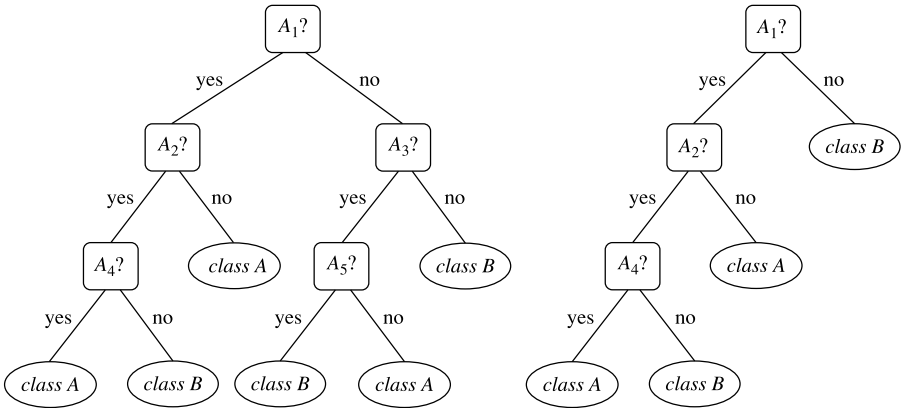


图 6.7 未剪枝的决策树（左）和其剪枝版本（右）。

然而，选择合适的阈值存在困难。高阈值可能导致树过于简化，而低阈值可能导致简化很少。

第二种更常见的方法是后剪枝，去除“完全生长”树的子树。在给定节点处，通过去除其分支并用叶子节点替换来剪枝子树。

叶子节点被标记为被替换子树中最频繁的分类标签。例如，请注意图6.7中未剪枝树的节点“ A_3 ?”处的子树。假设该子树中最常见的类别是“*class B*。”在剪枝后的树中，相关的子树被剪枝，替换为叶子节点“*class B*。”

CART中使用的成本复杂度剪枝算法是后剪枝方法的一个例子。

该方法将树的成本复杂度视为树中叶子数量和树的错误率的函数（其中错误率是树错误分类的元组百分比）。它从树的底部开始。对于每个内部节点 N ，它计算节点 N 处子树的成本复杂度，以及如果剪枝（即替换为叶子节点）时节点 N 处子树的成本复杂度。将这两个值进行比较。如果在节点 N 处剪枝子树会导致更小的成本复杂度，则剪枝该子树；否则，保留它。

一个带类别标签的元组剪枝集用于估计成本复杂度。该集合独立于（1）用于构建未剪枝树的训练集和（2）用于准确性估计的任何测试集。

该算法生成一组逐步修剪的树。一般来说，最小化成本复杂度的最小决策树是首选。

C4.5使用一种称为悲观修剪的方法，这种方法与成本复杂度方法类似，因为它也使用错误率估计来做出关于子树修剪的决策。然而，悲观修剪不需要使用修剪集。相反，它使用训练集来估计错误率。

请记住，基于训练集的准确性或错误率估计过于乐观，因此存在很强的偏差。因此，悲观修剪方法通过添加惩罚来调整从训练集中获得的错误率，以抵消所产生的偏差。

我们可以根据编码所需的位数来修剪树，而不是基于估计的错误率。“最佳”修剪树是指最小化编码位数的树。这种方法采用了在第6.2.2节中简要介绍的MDL原则。基本思想是优先选择最简单的解决方案。与成本复杂度修剪不同，它不需要独立的元组集（即修剪集）。

另外，预剪枝和后剪枝可以交替进行，以实现综合方法。后剪枝所需的计算量大于预剪枝，但通常会导致更可靠的树。没有一种剪枝方法被发现优于所有其他方法。尽管某些剪枝方法确实依赖于额外数据的可用性，但在处理大型数据库时，这通常不是一个問題。

尽管剪枝后的树通常比未剪枝的树更紧凑，但它们仍然可能相当大且复杂。决策树可能会遭受重复和复制（图6.8），使其难以解释。重复发生在树的某个分支上反复测试某个属性时（例如，“年龄 < 60?”接着是“年龄 < 45?”等等）。在复制中，树内存在重复的子树。这些情况可能会妨碍决策树的准确性和可理解性。使用多变量分裂（基于属性组合的分裂）可以防止这些问题。另一种方法是使用不同形式的知识表示，例如规则，而不是决策树。这在第7章中进行了描述，展示了如何通过从决策树中提取IF-THEN规则来构建基于规则的分类器。

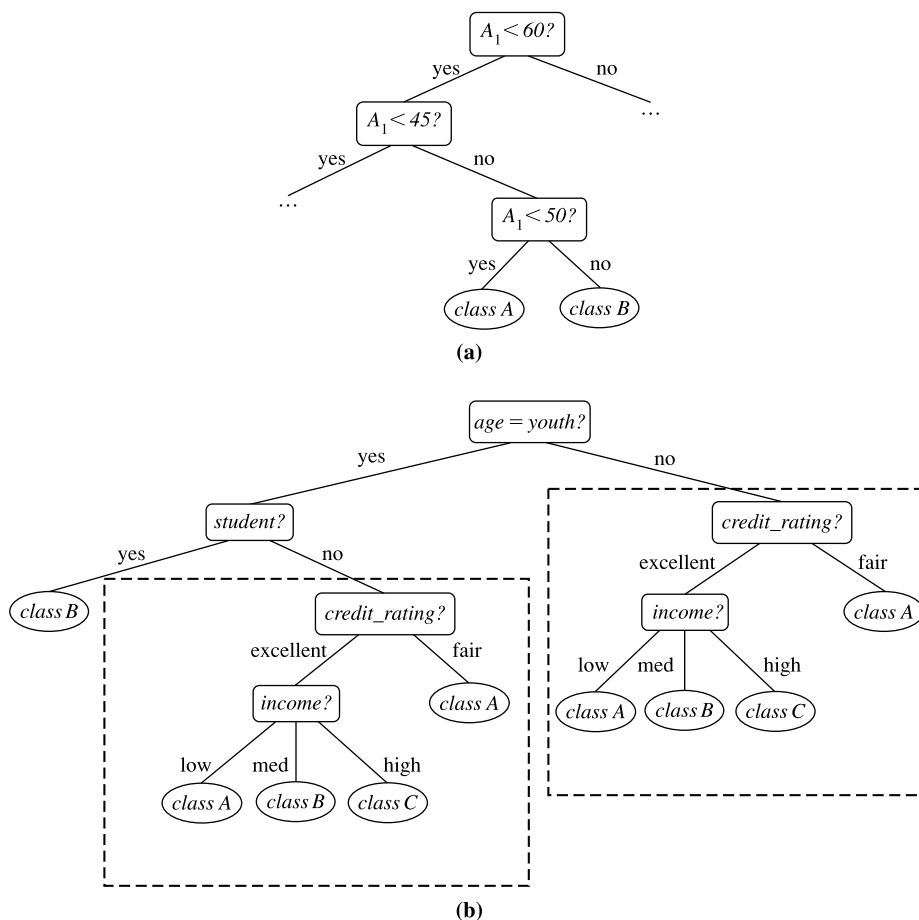


图6.8

(a) 子树重复的示例，其中一个属性在树的给定分支上被反复测试（例如，年龄），以及 (b) 子树复制的示例，其中树中存在重复的子树（例如，以节点“信用评级?”为首的子树）。

6.3 贝叶斯分类方法

“贝叶斯分类器是什么？”贝叶斯分类器是统计分类器。它们可以预测类别成员概率，例如，给定元组属于特定类别的概率。

贝叶斯分类基于贝叶斯定理，接下来将进行描述。比较分类算法的研究发现，一种称为 *天真的贝叶斯分类器* 的简单贝叶斯分类器在性能上与决策树和选定的神经网络分类器相当。贝叶斯分类器在应用于大型数据库时也表现出高准确性和速度。

天真的贝叶斯分类器假设一个属性值对给定类别的影响与其他属性的值是独立的。这个假设被称为 **类条件独立性**。作出这个假设是为了简化计算，因此在这个意义上被认为是“天真的”。

第6.3.1节回顾基本概率符号和贝叶斯定理。在第6.3.2节中，您将学习如何进行朴素贝叶斯分类。

6.3.1 贝叶斯定理

贝叶斯定理以托马斯·贝叶斯（Thomas Bayes）的名字命名，他是一位不拘一格的英国牧师，在18世纪早期从事概率和决策理论的研究。设 X 为一个数据元组。在贝叶斯的术语中， X 被视为“证据”。通常，它是通过对一组 n 属性进行测量来描述的。设 H 为某个假设，例如数据元组 X 属于指定类别 C 。对于分类问题，我们想要确定 $P(H|X)$ ，即在给定“证据”或观察到的数据元组 X 的情况下，假设 H 成立的概率。换句话说，我们正在寻找元组 X 属于类别的概率。

C ，考虑到我们知道 X 的属性描述。

$P(H|X)$ 是 H 在 X 条件下的后验概率，或称为后验概率。例如，假设我们的数据元组世界仅限于由属性年龄和收入描述的客户，并且 X 是一位35岁的客户，年收入为\$40,000。假设 H 是我们的客户会购买电脑的假设。那么 $P(H|X)$ 反映了在我们知道客户的年龄和收入的情况下，客户 X 会购买电脑的概率。

相对而言， $P(H)$ 是 H 的先验概率，或称为先验概率。以我们的例子来说，这是任何给定客户购买计算机的概率，无论年龄、收入或其他任何信息。后验概率 $P(H|X)$ 是基于比先验概率 $P(H)$ 更多的信息（例如，客户信息），而先验概率 $P(H)$ 与 X 无关。

类似地， $P(X|H)$ 是条件概率，表示在 H 的条件下 X 的概率。也就是说，给定我们知道客户会购买计算机，客户 X 是35岁且年收入40,000美元的概率。在分类中， $P(X|H)$ 通常也被称为似然性。

$P(X)$ 是 X 的先验概率。以我们的例子来说，这是我们客户集中一个人35岁且年收入40,000美元的概率。在分类中， $P(X)$ 通常也被称为边际概率。

“这些概率是如何估计的？” $P(H)$ 、 $P(X|H)$ 和 $P(X)$ 可以从给定的数据中估计，正如我们接下来将看到的。贝叶斯定理的作用在于它提供了一种从 $P(H)$ 、 $P(X|H)$ 和 $P(X)$ 计算后验概率 $P(H|X)$ 的方法。贝叶斯定理是

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}. \quad (6.11)$$

“贝叶斯分类器是什么样的？”假设有 m 个类别， $C_1, C_2, \dots, C_m$ 。给定一个元组， X ，我们想要预测它属于哪个类别。在贝叶斯分类器中，它首先计算每个 m 类别的后验概率， $P(C_i|X)$ ($i=1, \dots, m$)，然后预测元组 X 属于后验概率最高的类别。在上述例子中，给定一个35岁、年收入\$40,000的客户 X ，我们想要预测该客户是否会购买电脑。因此，在这个任务中，有两个可能的类别（购买电脑与不购买电脑）。假设 $P(\text{购买电脑}|X)=0.8$ 和 $P(\text{不购买电脑}|X)=0.2$ 。贝叶斯分类器将预测客户 X 会购买电脑。

“那么，贝叶斯分类器的效果如何？”从理论上讲，贝叶斯分类器是最优的，因为它的分类错误率是所有其他分类器中最小的。由于贝叶斯分类器是一种概率方法，它可能会对任何给定的元组做出错误的预测。在上述例子中，贝叶斯分类器预测客户会购买电脑。由于 $P(\text{不购买电脑} | X) = 0.2$ ，因此贝叶斯分类器做出错误预测的概率为20%。然而，由于贝叶斯分类器总是预测具有最大后验概率的类别，因此其对给定元组 X （通常称为风险）的错误预测概率在所有其他分类器中是最低的。

在我们的例子中，给定客户的风险为0.2。换句话说，贝叶斯分类器的预测错误的概率为20%。因此，贝叶斯分类器的整体分类错误率，即所有可能元组风险的期望（即加权平均），在所有可能的分类器中是最低的。鉴于其理论上的最优性，贝叶斯分类器在统计机器学习领域中扮演着基础性的角色。例如，许多分类器（例如，朴素贝叶斯分类器，

k -最近邻分类器、逻辑回归、贝叶斯网络等可以被视为近似的贝叶斯分类器。贝叶斯分类器的一个优点在于，它为其他不明确使用贝叶斯定理的分类器提供了理论依据。例如，在某些假设下，可以证明许多神经网络和曲线拟合算法输出的最大后验假设与贝叶斯分类器相同。

“那么，我们为什么不直接使用贝叶斯分类器呢？”根据贝叶斯定理（公式（6.11）），为了计算后验概率 $P(C_i | X)$ ($i = 1, \dots, m$)，我们需要知道条件概率 $P(X | C_i)$ ($i = 1, \dots, m$)、先验概率 $P(C_i)$ ($i = 1, \dots, m$) 和边际概率 $P(X)$ 。在贝叶斯分类器中，我们只需要知道哪个类别具有最高的后验概率，对于给定的元组 X ，其边际概率与不同类别是独立的。换句话说，不同的后验概率 $P(C_i | X)$ ($i = 1, \dots, m$) 共享相同的边际概率 $P(X)$ 。

因此，为了预测给定元组属于哪个类别，我们只需要估计条件概率 $P(X | C_i)$ ($i = 1, \dots, m$)，以及先验概率 $P(C_i)$ ($i = 1, \dots, m$)。

从训练数据集中估计先验概率 $P(C_i)$ ($i = 1, \dots, m$) 相对容易（具体细节将在下一节介绍）。另一方面，直接估计条件概率 $P(X | C_i)$ ($i = 1, \dots, m$) 通常非常具有挑战性。为了说明这一点，假设有 n 个二元属性 $A_1, A_2, \dots, A_n$ 。那么， n 维属性向量 X 有 2^n 个可能值，我们需要估计每个可能的属性向量值相对于每个类别标签的条件概率。⁶换句话说，属性值空间是指数级的！估计如此大量的条件概率参数是非常困难的。⁷

因此，贝叶斯分类器的主要困难在于如何有效地估计条件概率，通常需要一些近似。已经开发了许多解决方案。其中一种努力，可能是最简单但相当有效的解决方案，就是朴素贝叶斯分类器，我们将在接下来介绍。

⁵ 边际概率 $P(X)$ 本身可以基于条件概率 $P(X | C_i)$ ($i = 1, \dots, m$) 进行计算，

先验 $P(C_i)$ ($i = 1, \dots, m$) 基于全概率法则，即， $P(X) =$

$\sum_{i=1}^m P(X | C_i) P(C_i)$. 在某些情况下（

例如，估计贝叶斯分类器的风险），有必要计算边际概率。

⁶ 我们需要估计的条件概率的参数总数为 $m(2^n - 1)$ 。具体细节如下：留作练习。

⁷ 在统计学中，这意味着估计结果具有高方差，因而不可靠。

6.3.2 天真贝叶斯分类

朴素贝叶斯分类器，或称简单贝叶斯分类器，遵循与贝叶斯分类器相同的程序，除了它估计条件概率的方式不同。具体来说，它的工作原理如下：

1. 设 D 为一组元组及其相关的类别标签的训练集。与往常一样，每个元组由一个 n 维属性向量表示， $X=(x_1, x_2, \dots, x_n)$ ，描绘了从 n 个属性 $A_1, A_2, \dots, A_n$ 上对元组进行的 n 次测量。
2. 假设有 m 个类别， $C_1, C_2, \dots, C_m$ 。给定一个元组， X ，分类器将预测 X 属于具有最高后验概率的类别，条件是 X 。也就是说，朴素贝叶斯分类器预测元组 X 属于类别 C_i ，当且仅当

$$P(C_i|X) > P(C_j|X) \quad \text{for } 1 \leq j \leq m, j \neq i.$$

因此我们最大化 $P(C_i|X)$ 。对于最大化 $P(C_i|X)$ 的类别 C_i 称为最大后验假设。根据贝叶斯定理（公式 (6.11)），

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}. \quad (6.12)$$

3. 由于 $P(X)$ 对所有类别都是常数，我们只需要找出哪个类别最大化 $P(X|C_i)P(C_i)$ 。如果类别的先验概率未知，则通常假设各类别是等可能的，即 $P(C_1)=P(C_2)=\dots=P(C_m)$ ，因此我们将最大化 $P(X|C_i)$ 。否则，我们最大化 $P(X|C_i)P(C_i)$ 。注意，类别的先验概率可以通过 $P(C_i)=|C_{i,D}|/|D|$ 来估计，其中 $|C_{i,D}|$ 是数据集 D 中类别 C_i 的训练元组数量。
4. 给定一个具有多个属性的数据集，计算将会非常耗费计算资源 $P(X|C_i)$ 是基于上述原因。为了减少在评估 $P(X|C_i)$ 时的计算，假设了类条件独立性。这假设属性值在给定元组标签的情况下是条件独立的（即，如果我们知道元组属于哪个类，则属性之间没有依赖关系）。因此 n

$$\begin{aligned} P(X|C_i) &= \prod_{k=1}^n P(x_k|C_i) \\ &= P(x_1|C_i) \times P(x_2|C_i) \times \dots \times P(x_n|C_i). \end{aligned} \quad (6.13)$$

我们可以很容易地从训练元组中估计概率 $P(x_1|C_i), P(x_2|C_i), \dots, P(x_n|C_i)$ 。请记住，这里 x_k 指的是元组 X 的属性 A_k 的值。对于每个属性，我们查看该属性是分类的还是连续值的。例如，为了计算 $P(X|C_i)$ ，我们考虑以下内容：

- a. 如果 A_k 是分类的，则 $P(x_k|C_i)$ 是在 D 中类 C_i 的元组中具有值 x_k 的元组数量，除以 $|C_{i,D}|$ ，即在 D 中类 C_i 的元组数量。⁸

⁸ 在统计学中，这就是经典的极大似然估计（MLE）方法。

- b. 如果 A_k 是连续值的, 那么我们需要做更多的工作, 但计算相当简单。通常假设连续值属性具有高斯分布, 其均值为 μ 和标准差为 σ , 定义为

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad (6.14)$$

以便

$$P(x_k|C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i}). \quad (6.15)$$

这些方程可能看起来令人生畏, 但请稍等! 我们需要计算 μ_{C_i} 和 σ_{C_i} , 它们分别是类 C_i 的训练元组中属性 A_k 值的均值(即平均值)和标准差。然后, 我们将这两个量代入公式(6.14), 再加上

x_k , 以估计 $P(x_k|C_i)$ 。

例如, 设 $X=(35, \$40,000)$, 其中 A_1 和 A_2 分别是属性 age 和 $income$, 类标签属性为 $buys_computer$ 。与 X 相关的类标签是 yes (即, $buys_computer=yes$)。假设 age 尚未离散化, 因此作为连续值属性存在。假设从训练集中, 我们发现购买计算机的客户在 D 中的年龄为 38 ± 12 岁。换句话说, 对于属性 age 和这个类(即, $buys_computer=yes$), 我们有 $\mu=38$ 岁, $\sigma=12$ 。我们可以将这些量与 $x_1=35$ 代入公式(6.14), 以估计 $P(age=35|buys_computer=yes)$ 。有关均值和标准差计算的快速回顾, 请参见第2.2节。

5. 为了预测 X 的类别标签, $P(X|C_i)P(C_i)$ 会针对每个类别 C_i 进行评估。分类器预测当且仅当元组 X 的类别标签为类别 C_i 时

$$P(X|C_i)P(C_i) > P(X|C_j)P(C_j) \quad \text{for } 1 \leq j \leq m, j \neq i. \quad (6.16)$$

换句话说, 预测的类别标签是类别 C_i , $P(X|C_i)P(C_i)$ 的值是最大的。

“朴素贝叶斯分类器的有效性如何?” 注意到朴素贝叶斯分类器与贝叶斯分类器之间唯一的区别是类别条件独立假设。因此, 如果这样的假设确实成立, 朴素贝叶斯分类器将是最优的, 具有最小的可能分类错误。然而, 在实践中, 这并不总是如此, 因为在使用时所做的假设(如类别条件独立性)存在不准确性, 以及缺乏可用的概率数据。尽管如此, 关于该分类器与决策树和选定神经网络分类器的各种实证研究发现, 在某些领域它是可比的。朴素贝叶斯分类器的另一个优点是它可以自然地处理缺失的属性。

示例 6.5. 使用朴素贝叶斯分类进行类别标签预测。我们希望使用朴素贝叶斯分类来预测一个元组的类别标签, 给定与示例6.3中决策树归纳相同的训练数据。训练数据在表6.1中已显示。数据元组由属性 age 、 $income$ 、 $student$ 和 $credit_rating$ 描述。类别标签属性 $buys_computer$ 有两个不同的值(即, $\{yes, no\}$)。让 C_1 对应于类别 $buys_computer=yes$, C_2 对应于 $buys_computer=no$ 。我们希望分类的元组是

$$X = (age = youth, income = medium, student = yes, credit_rating = fair).$$

我们需要找出哪个类别最大化 $P(X|C_i)P(C_i)$ ，对于 $i=1, 2$ 。 $P(C_i)$ 是每个类别的先验概率可以根据训练元组计算得出：

$$P(\text{buys_computer} = \text{yes}) = 9/14 = 0.643$$

$$P(\text{buys_computer} = \text{no}) = 5/14 = 0.357.$$

为了计算 $P(X|C_i)$ ，对于 $i=1, 2$ ，我们计算以下条件概率：

$$P(\text{age} = \text{youth} | \text{buys_computer} = \text{yes}) = 2/9 = 0.222$$

$$P(\text{age} = \text{youth} | \text{buys_computer} = \text{no}) = 3/5 = 0.600$$

$$P(\text{income} = \text{medium} | \text{buys_computer} = \text{yes}) = 4/9 = 0.444$$

$$P(\text{income} = \text{medium} | \text{buys_computer} = \text{no}) = 2/5 = 0.400$$

$$P(\text{student} = \text{yes} | \text{buys_computer} = \text{yes}) = 6/9 = 0.667$$

$$P(\text{student} = \text{yes} | \text{buys_computer} = \text{no}) = 1/5 = 0.200$$

$$P(\text{credit_rating} = \text{fair} | \text{buys_computer} = \text{yes}) = 6/9 = 0.667$$

$$P(\text{credit_rating} = \text{fair} | \text{buys_computer} = \text{no}) = 2/5 = 0.400.$$

使用这些概率，我们得到

$$\begin{aligned} P(X|\text{buys_computer} = \text{yes}) &= P(\text{age} = \text{youth} | \text{buys_computer} = \text{yes}) \\ &\quad \times P(\text{income} = \text{medium} | \text{buys_computer} = \text{yes}) \\ &\quad \times P(\text{student} = \text{yes} | \text{buys_computer} = \text{yes}) \\ &\quad \times P(\text{credit_rating} = \text{fair} | \text{buys_computer} = \text{yes}) \\ &= 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044. \end{aligned}$$

类似地，

$$P(X|\text{buys_computer} = \text{no}) = 0.600 \times 0.400 \times 0.200 \times 0.400 = 0.019.$$

为了找到最大化 $P(x|C_i)P(C_i)$ 的类别 c_i ，我们计算

$$P(X|\text{buys_computer} = \text{yes})P(\text{buys_computer} = \text{yes}) = 0.044 \times 0.643 = 0.028$$

$$P(X|\text{buys_computer} = \text{no})P(\text{buys_computer} = \text{no}) = 0.019 \times 0.357 = 0.007.$$

因此，朴素贝叶斯分类器预测 $\text{buys_computer}=\text{yes}$ 对于元组 X 。 □

“如果我遇到零的概率值怎么办？”回想一下在公式 (6.13) 中，我们将 $P(X|C_i)$ 估计为 $P(x_1|C_i), P(x_2|C_i), \dots, P(x_n|C_i)$ 的乘积，基于类别条件独立的假设。这些概率可以从训练元组中估计（步骤 4）。我们需要计算每个类别的 $P(X|C_i)$ ($i=1, 2, \dots, m$)，以找到 $P(X|C_i)P(C_i)$ 最大化的类别 C_i （步骤 5）。让我们考虑这个计算。对于元组 X 中的每个属性-值对（即 $A_k=x_k$ ，对于 $k=1, 2, \dots, n$ ），我们需要计算每个类别（即每个 C_i ，对于 $i=1, \dots, m$ ）中具有该属性-值对的元组数量。在示例 6.5 中，我们有两个类别 ($m=2$)，即 $\text{buys_computer}=\text{yes}$ 和 $\text{buys_computer}=\text{no}$ 。因此，对于元组 X 的属性-值对 $\text{student}=\text{yes}$ ，假设，

我们需要两个计数——学生客户的数量，以及对于 $buys_computer=yes$ （这有助于 $P(X|buys_computer=yes)$ ）的计数，以及学生客户的数量和对于 $buys_computer=no$ （这有助于 $P(X|buys_computer=no)$ ）。

然而，如果说，对于类 $buys_computer=no$ ，没有代表学生的训练元组，导致 $P(student=yes|buys_computer=no)=0$ ，会发生什么呢？换句话说，如果我们最终得到某些 $P(x_k|C_i)$ 的概率值为零，会发生什么？将这个零值代入公式 (6.13) 将返回 $P(X|C_i)$ 的零概率，即使在没有零概率的情况下，我们可能会得到一个高概率，表明 X 属于类 C_i ！零概率抵消了所有其他（后验）概率（关于 C_i ）在乘积中的影响。

有一个简单的技巧可以避免这个问题。我们可以假设我们的训练数据库 D 是如此之大，以至于在每个计数上加一只会对估计的概率值产生微不足道的影响，但可以方便地避免零概率值的情况。这种概率估计技术被称为 拉普拉斯修正或 拉普拉斯估计器，以纪念法国数学家皮埃尔·拉普拉斯（Pierre Laplace），他生活在1749年至1827年。⁹ 如果我们有，比如说， q 个计数，我们在每个计数上加一，那么我们必须记得在概率计算中相应的分母上加上 q 。我们在示例6.6中说明了这一技术。

示例 6.6. 使用拉普拉斯校正来避免计算零概率值。假设在某个包含 1000 个元组的训练数据库 D 中，对于类别 $buys_computer=yes$ ，我们有 0 个元组的 $income=low$ ，990 个元组的 $income=medium$ ，和 10 个元组的 $income=high$ 。这些事件的概率，在没有拉普拉斯校正的情况下，分别为 0、0.990（来自 990/1000）和 0.010（来自 10/1000）。使用拉普拉斯校正后，我们假装每个收入值对都有一个额外的元组。这样，我们得到以下概率（四舍五入到小数点后三位）：

$$\frac{1}{1003} = 0.001, \frac{991}{1003} = 0.988, \text{ and } \frac{11}{1003} = 0.011,$$

分别。“校正”后的概率估计接近于“未校正”的对应值，但避免了零概率值。

□

朴素贝叶斯分类器的主要思想在于类条件独立假设，这显著简化了条件概率的估计 $P(X|C_i)$ ($i=1, \dots, m$)。

然而，这个（类条件独立假设）也是朴素贝叶斯分类器的一个主要限制，因为在某些应用中可能不成立。为了解决这个问题，我们需要更复杂的方法来近似条件概率，例如贝叶斯网络，这将在下一章中介绍。

⁹ 在统计学中，这属于最大后验估计（MAP）方法。这也可以被视为一种平滑技术（即，“平滑”零概率）。在实践中，我们还可以用一个小整数 k 替代1。直观上，我们有 k （而不是1）个额外的元组用于每个属性-值对。

6.4 懒惰学习者（或向邻居学习）

本书迄今讨论的分类方法——决策树归纳和贝叶斯分类——都是渴望学习者的例子。渴望学习者在给定一组训练元组时，会在接收新的（例如，测试）元组进行分类之前构建一个概括（即，分类）模型。我们可以认为学习到的模型已经准备好并渴望对之前未见过的元组进行分类。

想象一种对比鲜明的懒惰方法，其中学习者在进行任何模型构建以分类给定测试元组之前，反而等到最后一刻。也就是说，当给定一个训练元组时，懒惰学习者只是存储它（或仅进行一些小的处理），并等待给定一个测试元组。

只有在看到测试元组时，它才会执行概括，根据与存储的训练元组的相似性对元组进行分类。与渴望学习方法不同，懒惰学习者在呈现训练元组时工作较少，而在进行分类或数值预测时工作较多。由于懒惰学习者存储训练元组或“实例”，它们也被称为基于实例的学习者，尽管所有学习本质上都是基于实例的。

在进行分类或数值预测时，惰性学习者可能会在计算上非常昂贵。它们需要高效的存储技术，并且非常适合在并行硬件上实现。惰性学习者对数据结构的解释或洞察力很少。然而，惰性学习者自然支持增量学习。它们能够建模复杂的决策空间，这些空间具有超多边形形状，可能不如其他学习算法（例如由决策树建模的超矩形形状）那样容易描述。在本节中，我们将介绍两个惰性学习者的例子：k-近邻分类器（第6.4.1节）和基于案例的推理分类器（第6.4.2节）。

6.4.1 k-近邻分类器

k-近邻方法最早在20世纪50年代初被描述。当给定一个大型训练集时，该方法劳动密集，并且直到20世纪60年代计算能力提高后才开始流行。此后，它在模式识别领域得到了广泛应用。

假设你想决定是否应该购买一台电脑。你会怎么做？一种可能的决策方式是了解你朋友对此的决定（是否购买电脑）。如果你大多数亲密朋友都购买了电脑，也许你也会决定购买一台电脑。最近邻分类器遵循非常相似的类比学习思想，即通过将给定的测试元组与相似的训练元组进行比较来学习。训练元组由n个属性描述。每个元组代表n维空间中的一个点。通过这种方式，所有训练元组都存储在n维属性空间中。当给定一个未知元组时，k-近邻分类器在属性空间中搜索与未知元组最接近的k个训练元组（即，在上述例子中找到你的亲密朋友）。这k个训练元组是未知元组的k个“最近邻”。然后，k-近邻分类器选择k个最近邻中最常见的类标签作为未知元组的预测类标签（即，遵循上述例子中你朋友的多数决定）。，以遵循上述例子中你朋友的多数决定。

“接近度”是通过距离度量来定义的，例如欧几里得距离。两个点或元组之间的欧几里得距离，例如， $X_1 = (x_{11}, x_{12}, \dots, x_{1n})$ 和 $X_2 = (x_{21}, x_{22}, \dots, x_{2n})$ ，是

$$\text{dist}(X_1, X_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}. \quad (6.17)$$

换句话说，对于每个数值属性，我们取元组 X_1 和元组 X_2 中该属性对应值之间的差，平方这个差，并累加它。然后取总累积距离的平方根。通常，在使用公式 (6.17) 之前，我们会对每个属性的值进行归一化。这有助于防止初始范围较大的属性（例如，收入）在权重上超过初始范围较小的属性（例如，二元属性）。例如，可以使用最小-最大归一化将数值属性 A 的值 v 转换为范围 $[0, 1]$ 内的 v' ，通过计算

$$v' = \frac{v - \min_A}{\max_A - \min_A}, \quad (6.18)$$

其中 $\min A$ 和 $\max A$ 是属性 A 的最小值和最大值。第二章描述了其他数据归一化的方法，作为数据转换的一种形式。

对于 k 最近邻分类，未知元组被分配给其 k 个最近邻中最常见的类标签。当 $k=1$ 时，未知元组被分配给在属性空间中与其最近的训练元组的类。当 $k>1$ 时，我们可以对其 k 个最近邻的类标签进行（加权）多数投票。最近邻分类器也可以用于数值预测，即为给定的未知元组返回一个实值预测。在这种情况下，分类器返回与未知元组的 k 个最近邻相关的实值标签的（加权）平均值。

“但是如何计算非数值而是名义（或分类）属性的距离，例如颜色呢？”之前的讨论假设描述元组的属性都是数值型的。对于名义属性，一种简单的方法是比较元组 X_1 中该属性的对应值与元组 X_2 中的值。如果两者相同（例如，元组 X_1 和 X_2 都为蓝色），则两者之间的差异被视为 0。如果两者不同（例如，元组 X_1 是蓝色而元组 X_2 是红色），则差异被视为 1。其他方法可能会采用更复杂的差异评分方案（例如，对于蓝色和白色的差异分数可能大于蓝色和黑色的差异分数）。

“缺失值怎么办？”一般来说，如果在元组 X_1 或元组 X_2 中某个属性 A 的值缺失，我们假设最大可能的差异。假设每个属性都被映射到范围 $[0, 1]$ 。对于名义属性，如果 A 的对应值缺失一个或两个，我们将差异值视为 1。如果 A 是数值型并且在元组 X_1 和 X_2 中都缺失，则差异也视为 1。如果只有一个值缺失，另一个值（我们称之为 v' ）存在且已归一化，则我们可以将差异视为 $|1-v'|$ 或 $0-v'|$ （即 $1-v'$ 或 v' ），取较大者。

“我如何确定 k （邻居的数量）的良好值？”这可以通过实验来确定。从 $k=1$ 开始，我们使用测试集来估计分类器的错误率。这个过程可以每次通过增加 k 来重复，以允许许多一个邻居。可以选择给出最小错误率的 k 值。一般来说，训练元组的数量越大， k 的值也会越大（以便分类和数值预测决策可以基于更大部分的存储元组）。当训练元组的数量接近无穷大且 $k=1$ 时，错误率不会比贝叶斯错误率高出两倍（后者是理论上的最小值）。在

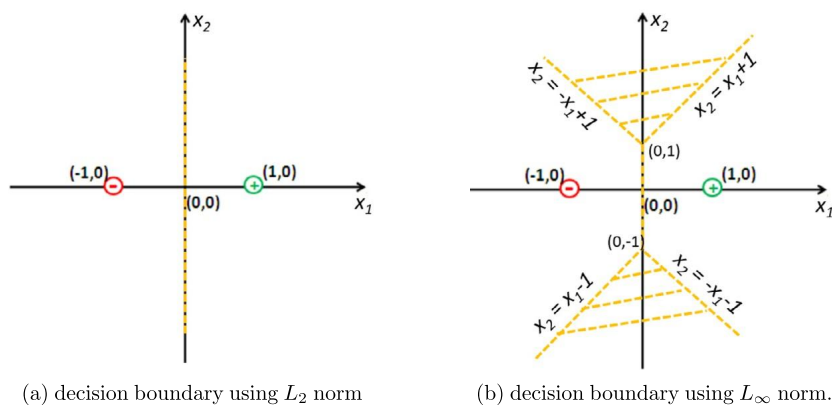


图 6.9

距离度量对1-最近邻分类器的影响。给定两个训练示例，包括一个位于 $(1,0)$ 的正例和一个位于 $(-1,0)$ 的负例。使用不同距离度量的1-最近邻分类器的决策边界彼此之间差异很大。使用 L_2 范数（左侧），决策边界是位于 $x_2 = 0$ 的垂直线。使用 L_∞ 范数（右侧），决策边界包括一段线段在

$(0,-1)$ 和 $(0,1)$ 以及两个阴影区域。

换句话说，1-最近邻分类器在渐近上是近似最优的。如果 k 趋近于无穷大，错误率趋近于贝叶斯错误率。

最近邻分类器使用基于距离的比较，这本质上对每个属性赋予相等的权重。因此，当给定噪声或无关属性时，它们可能会遭受较差的准确性。然而，该方法已被修改以纳入属性加权和噪声数据元组的剪枝。距离度量的选择可能至关重要。曼哈顿（城市街区）距离（第2.3节）或其他距离测量也可以使用。图6.9以示例的方式展示了距离度量对 k -最近邻分类器决策边界的影响。

在对测试元组进行分类时，最近邻分类器可能非常缓慢。如果 D 是一个包含 $|D|$ 个元组的训练数据库，并且 $k=1$ ，则需要 $O(|D|)$ 次比较来分类给定的测试元组。通过预排序并将存储的元组排列成搜索树，可以将比较次数减少到 $O(\log(|D|))$ 。并行实现可以将运行时间减少到一个常数，即 $O(1)$ ，独立于 $|D|$ 。

加速分类时间的其他技术包括使用部分距离 计算和编辑存储的元组。在部分距离方法中，我们基于 n 属性的子集计算距离。如果该距离超过阈值，则停止对给定存储元组的进一步计算，过程转向下一个存储元组。编辑方法删除被证明无用的训练元组。该方法也称为剪枝或压缩，因为它减少了存储的元组总数。加速最近邻搜索的另一种技术是通过局部敏感哈希 (LSH)。其关键思想是通过保持局部性的哈希函数将相似的元组以高概率哈希到同一个桶中。然后，给定一个测试元组，我们首先识别它属于哪个桶，然后仅在同一个桶中搜索训练元组以识别其最近邻。

6.4.2 基于案例的推理

基于案例的推理 (CBR) 分类器使用问题解决方案的数据库来解决新问题。

与 k -最近邻分类器不同，后者将训练元组存储为欧几里得空间中的点，CBR将元组或“案例”作为复杂的符号描述进行问题解决。CBR的商业应用包括客户服务帮助台的问题解决，其中案例描述与产品相关的诊断问题。CBR还应用于工程和法律等领域，其中案例分别是技术设计或普通法系统中的法律裁决。

医学教育是案例推理 (CBR) 的另一个领域，在这个领域中，患者的病历和治疗方案被用来帮助诊断和治疗新患者。

在给定一个新的案例进行分类时，案例推理系统首先会检查是否存在一个相同的训练案例。如果找到一个相同的案例，则返回该案例的解决方案。如果没有找到相同的案例，案例推理系统将搜索具有与新案例相似的组成部分的训练案例。从概念上讲，这些训练案例可以被视为新案例的邻居。如果案例被表示为图形，这涉及到搜索与新案例中的子图相似的子图。案例推理系统尝试结合邻近训练案例的解决方案，以提出新案例的解决方案。如果个别解决方案之间出现不兼容的情况，则可能需要回溯以寻找其他解决方案。案例推理系统可能会利用背景知识和问题解决策略来提出可行的综合解决方案。

案例推理中的关键挑战包括找到良好的相似性度量（例如，用于匹配子图）和合适的解决方案组合方法。其他挑战包括选择用于索引训练案例的显著特征以及开发高效的索引技术。随着存储案例数量的增加，准确性和效率之间的权衡变得越来越重要。

随着这个数字的增加，基于案例的推理器变得更加智能。然而，在某个点之后，系统的效率将受到影响，因为搜索和处理相关案例所需的时间会增加。与最近邻分类器一样，一个解决方案是编辑训练数据库。为了提高性能，冗余的案例或那些未被证明有用的案例可能会被丢弃。然而，这些决策并不是明确的，它们的自动化仍然是一个活跃的研究领域。

6.5 线性分类器

到目前为止，我们已经学习了一些能够生成复杂决策边界的分类器。例如，决策树分类器可能输出一个超矩形形状的决策边界（图 6.10 (a)），而 k 最近邻分类器可能输出一个超多边形形状的决策边界（图 6.10 (b)）。但是，简单的线性决策边界呢？在图 6.10 的例子中，直观上，线性决策边界（图 6.10 (c) 中的直线）在将正训练元组与负训练元组分开时（几乎）与决策树分类器和 k 最近邻分类器一样好。然而，这样的线性决策边界可能提供额外的优势，例如高效的分类器训练计算、更好的泛化性能和更好的可解释性。

在本节中，我们介绍学习线性分类器的基本技术。我们将从线性回归开始，它构成了线性分类器的基础。然后，我们将介绍两种线性分类器。

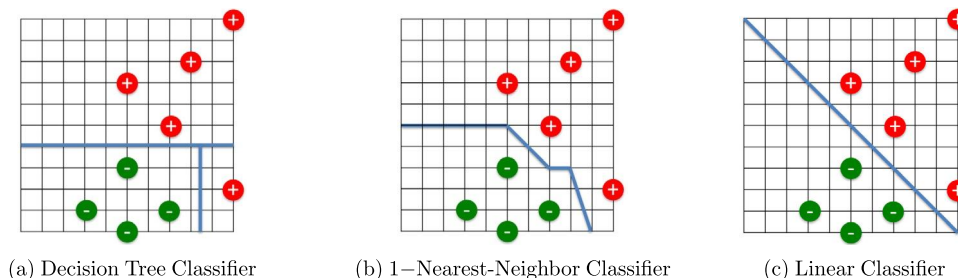


图 6.10

不同分类器的决策边界。请注意，这个例子是线性可分的，这意味着线性分类器 (c) 可以完美地将所有正训练样本与所有负训练样本分开。如果训练集是线性不可分的，我们仍然可以使用线性分类器，但代价是一些训练样本位于决策边界的“错误”一侧。在第7章中，我们将介绍处理线性不可分情况的技术（例如，支持向量机）。

分类器，包括 (1) 感知机，它是最早的线性分类器之一，以及 (2) 逻辑回归，它是最广泛使用的线性分类器之一。其他线性分类器将在第7章中介绍，例如线性支持向量机。

6.5.1 线性回归

线性回归是一种统计技术，它基于一个或多个自变量预测连续值。例如，我们可能想根据居住面积预测房价，或者根据学生所就读的大学、专业和整体 GPA 等预测未来收入。由于线性回归旨在预测连续值，因此不能直接应用于分类任务，其中输出是分类变量。尽管如此，线性回归中的核心技术构成了线性分类器的基础。因此，让我们首先简要介绍线性回归。

假设我们有 n 个元组，每个元组由 p 个属性表示 $x_i = (x_{i,1}, \dots, x_{i,p})^T$ 和一个连续输出值 $y_i (i=1, \dots, n)$ 。在线性回归中，我们希望学习一个线性函数，将 p 个输入属性 x_i 映射到输出变量 y_i ，即 $\hat{y}_i = w^T x_i + b = \sum_{j=1}^p w_j x_{i,j} + b$ ，其中 $\hat{y}_i$ 是第 i 个元组的预测输出值， $w = (w_1, \dots, w_p)^T$ 是一个 p 维的权重向量， b 是偏置标量。换句话说，线性回归假设输出值是 p 个输入属性值的线性加权和，偏移量为偏置标量 b 。权重向量 $w_j (j=1, \dots, p)$ 中的条目表示对应属性 $x_{i,j}$ 在预测输出变量 y_i 中的重要性。在上述示例中，线性回归模型假设房价与居住面积呈线性相关；学生的未来收入可以通过她所就读的大学、专业和整体 GPA 的线性加权组合（加上一个偏置标量 b ）来预测。如果我们知道权重向量 w 和偏置标量 b ，便可以根据其 p 个输入属性值来预测输出值。

“那么，我们如何确定权重向量 w 和偏置标量 b ？”直观上，我们希望从训练数据中学习到“最佳”的权重向量 w 和“最佳”的偏置标量 b ，以便线性回归模型能够做出“最佳”的预测。也就是说，预测值 $\hat{y}_i = w^T x_i + b$ 尽可能接近实际观察值 $y_i (i=1, \dots, n)$ 。

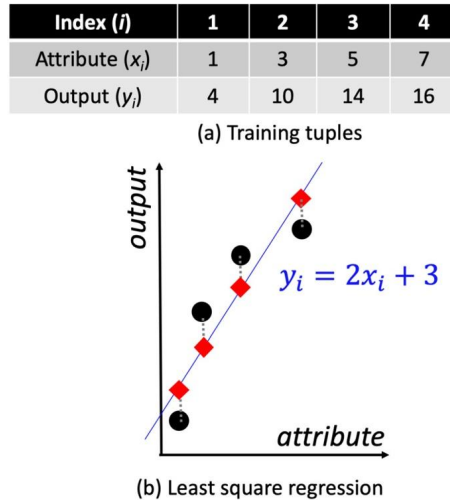


图 6.11

最小二乘回归的一个例子。(a) 四个训练元组。(b) 训练元组（黑点）和最小二乘回归模型（蓝线）的散点图。红色菱形是预测输出 $\hat{y}_i (i=1,2,3,4)$ ，虚线表示相应训练元组的预测误差 $(|y_i - \hat{y}_i|)$ 。

线性回归模型能够做出“最佳”的预测。也就是说，预测值 $\hat{y}_i = w^T x_i + b$ 尽可能接近实际观察值 $y_i (i=1, \dots, n)$ 。最常见的线性回归方法之一称为 最小二乘回归，旨在最小化以下损失函数 $L(w, b) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (w^T x_i + b))^2$ 。因此，最佳权重向量 w 和偏置标量 b 是最小化损失函数 $L(w, b)$ 的值，该函数衡量预测输出值 $\hat{y}_i$ 和实际观察值 y_i 之间的平方差之和。例如，如果只有一个输入属性（即， $p=1$ ），则最优权重 $w=$

$$\frac{\sum_{i=1}^n x_i (y_i - \bar{y})}{\sum_{i=1}^n x_i^2 - \frac{1}{n} (\sum_{i=1}^n x_i)^2} \text{ 且}$$

最优偏置标量 $b = \frac{1}{n} \sum_{i=1}^n (y_i - w x_i)$ ，其中 $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ 是所有 n 训练元组中观测输出值的平均值

示例 6.7. 让我们看一个最小二乘回归的例子，如图6.11所示。这里有四个训练元组，每个由一个一维属性 x_i 和一个输出变量 $y_i (i=1,2,3,4)$ 表示。

我们想要找到最小二乘回归模型 $y = wx + b$ ，基于输入属性 x 预测输出 y 。我们使用上述两个方程来找到最优权重 w 和最优偏置标量 b 。我们首先按如下方式找到最优权重 w 。四个训练元组的平均输出为 $\bar{y} = (y_1 + y_2 + y_3 + y_4) / 4 = (4 + 10 + 14 + 16) / 4 = 11$ 。因此我们有 $\sum_{i=1}^4 x_i (y_i - \bar{y}) = 1(4 - 11) + 3(10 - 11) + 5(14 - 11) + 7(16 - 11) = 40$ 。与此同时，我们有 $\sum_{i=1}^4 x_i^2 = 1^2 + 3^2 + 5^2 + 7^2 = 84$ 和 $1/4 (\sum_{i=1}^4 x_i)^2 = (1 + 3 + 5 + 7)^2 / 4 = 64$ 。

因此，最优权重 $w = \frac{\sum_{i=1}^n x_i (y_i - \bar{y})}{\sum_{i=1}^n x_i^2 - \frac{1}{n} (\sum_{i=1}^n x_i)^2} = \frac{40}{84 - 64} = 2$ 。基于最优权重 w ，最优偏置标量 $b = \frac{\sum_{i=1}^4 (y_i - w x_i)}{4} = \frac{(4 - 2 \times 1) + (10 - 2 \times 3) + (14 - 2 \times 5) + (16 - 2 \times 7)}{4} = 3$ 。 □

“但是，如果有多个 p ($p > 1$) 属性呢？”在这种情况下（称为多元线性回归），我们首先稍微改变一下符号。我们假设有一个额外的“虚拟”属性，对于任何元组，它的值始终为1。设这个虚拟属性的权重为 w_0 。则整体权重向量 $w = (w_0, w_1, \dots, w_p)$ ，新的输入属性向量 $x_i = (1, x_{i,1}, \dots, x_{i,p})$ 都是 $(p+1)$ 维向量。多元线性回归模型可以重写为 $\hat{y}_i = w^T x_i = w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}$ 。我们使用与之前相同的损失函数，即， $L(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (w^T x_i))^2$ 。结果表明，最优权重向量 w 可以计算为 $w = (X X^T)^{-1} X y$ ，其中 $X = [x_1, x_2, \dots, x_n]$ 是一个 $(p+1) \times n$ 矩阵，而 $y = [y_1, \dots, y_n]^T$ 是一个 $n \times 1$ 向量。（如何推导单线性回归和多线性回归的闭式解留作练习。）

在最小二乘回归中，我们通过预测值与实际输出值之间平方差的总和来衡量学习到的回归模型的“优度”。平方损失可能对训练集中的异常值敏感。在鲁棒回归中，它使用对这些异常值不太敏感的替代损失函数。例如，鲁棒回归中的Huber方法使用以下损失： $L(w) = \sum_{i=1}^n l_H(y_i - \hat{y}_i)$ ，其中 $l_H(y_i - \hat{y}_i) = (y_i - \hat{y}_i)^2$ 如果 $|y_i - \hat{y}_i| < \theta$ ， $l_H(y_i - \hat{y}_i) = 2\theta|y_i - \hat{y}_i| - \theta^2$ 否则，且 $\theta > 0$ 是用户指定的参数。注意，多线性回归的最优权重向量 w 涉及矩阵的逆（即 $(X X^T)^{-1}$ ）。如果 $p > n$ （即属性的数量超过训练元组的数量），则该矩阵的逆不存在。解决此问题的有效方法是引入一个关于权重向量 w 范数的正则化项。例如，如果我们使用 l_2 范数的权重向量 w ，相应的回归模型称为岭回归；如果我们使用 l_1 范数的权重向量 w ，相应的回归模型称为Lasso回归，它通常学习到一个稀疏的权重向量。这意味着学习到的权重向量 w 的某些条目为零，这表明这些属性在回归模型中未被使用。在第7.1节中，我们将使用Lasso回归进行特征选择。

6.5.2 感知器：将线性回归转化为分类

“我们如何修改线性回归模型以执行分类任务？”假设我们有一个二分类任务。¹⁰ 对于给定的元组，输出值 y_i 是一个二元变量： $y_i = +1$ 表示第一个元组是一个正元组（例如，购买电脑），而 $y_i = 0$ 表示第一个元组是一个负元组（例如，不购买电脑）。修改线性回归模型以适应这种二分类任务的一种方法是使用线性回归模型输出的符号作为预测的类别标签，即， $\hat{y}_i = \text{sign}(w^T x_i)$ ，其中 $\hat{y}_i$ 是第一个元组的预测类别标签， $\text{sign}(z) = 1$ 如果 $z > 0$ ， $\text{sign}(z) = 0$ 否则。请注意，我们使用与多元线性回归相同的符号，其中引入了一个“虚拟”属性，该属性对于任何元组始终取值为1。因此，如果我们知道权重向量 w ，我们可以用它来预测给定元组的类别标签，如下所示。我们计算给定元组的属性值的线性组合，权重由权重向量 w 的相应条目加权。如果这种线性组合的结果值为正，我们预测给定元组是一个正元组。否则，我们预测它是一个负元组。

“我们如何从一组训练元组中找到最优权重向量 w ？”训练感知器的经典学习算法如下。我们从权重向量 w 的初始猜测开始（例如，

¹⁰ 对于我们接下来将介绍的感知器和逻辑回归分类器，我们专注于二分类任务。然而，我们介绍的技术可以推广到处理这两种分类器的多类分类任务。

我们可以简单地设置 $w=0$ 。然后，学习算法将迭代，直到收敛，或者达到最大迭代次数或其他预设的停止标准。在每次迭代中，我们对每个训练元组 x_i 执行以下操作。我们尝试使用当前权重向量 w 预测 x_i 的类别标签，即， $\hat{y}_i = \text{sign}(w^T x_i)$ 。如果预测正确（即， $\hat{y}_i = y_i$ ），我们对权重向量不做任何处理。然而，如果预测不正确（即， $\hat{y}_i \neq y_i$ ），我们以以下两种方式之一更新当前权重向量。如果 $y_i = +1$ （即，第 i 个元组是正元组，但当前分类器预测它是负元组），我们将权重向量更新为 $w \leftarrow w + \eta x_i$ 。如果 $y_i = -1$ （即，第 i 个元组是负元组，当前分类器错误地预测为正元组），我们将权重向量更新为 $w \leftarrow w - \eta x_i$ ，其中 $\eta > 0$ 是用户指定的学习率。因此，直观上，在训练过程的每次迭代中，算法将专注于当前权重向量 w 错误预测的那些训练元组。如果错误预测的训练元组 x_i 是正元组，我们通过将权重向量 w 向该训练元组的属性向量 x_i 移动来更新权重向量（即，

$w \leftarrow w + \eta x_i$ ）。另一方面，如果错误预测的训练元组 x_i 是一个负元组，我们通过将权重向量 w 移离该训练元组的属性向量 x_i 来更新权重向量（即， $w \leftarrow w - \eta x_i$ ）。

例 6.8。让我们看一个图 6.12 的例子，用于训练感知器分类器。在图 6.12 中，我们假设偏置 $w_0 = 0$ 以便于说明。图 6.12(a)（左）显示了当前的决策边界和权重向量 w ，其中两个训练元组被错误分类，包括一个正元组 x_1 和一个负元组 x_8 。因此，只有这两个元组用于在当前迭代中更新权重向量，即， $w \leftarrow w + \eta x_1 - \eta x_8$ 。更新后的权重向量 w 和相应的决策边界如图 6.12(b)（右）所示，其中所有训练元组都被正确分类。

□

“感知器学习算法的有效性如何？”如果训练元组是线性可分的（例如，图 6.12 中的例子），则感知器算法保证找到一个权重向量（即，一个超平面决策边界），完美地将所有正训练元组与所有负训练元组分开。然而，如果训练元组不是线性可分的，该算法将无法收敛。

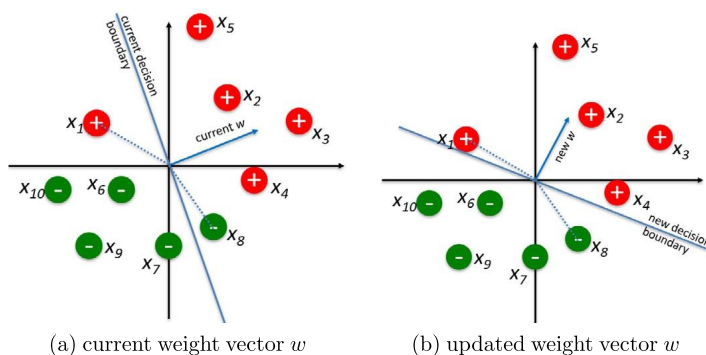


图 6.12

训练感知机分类器。

感知机是最早的线性分类器之一，最早在1958年被发明。它也可以作为深度神经网络中的一个构建块（称为“神经元”），将在第10章中介绍。

6.5.3 逻辑回归

我们在上一节中介绍的感知机能够预测给定元组的二元类别标签。然而，我们能否还判断这种预测的可信度？再一次，让我们考虑一个二元分类任务，并假设有两个可能的类别标签，即， $y = 1$ 表示正元组， $y = 0$ 表示负元组。回想一下，在（天真）贝叶斯分类器中，我们可以估计后验概率 $P(y_i = 1 | x_i)$ ，这可以直接用来指示预测分类结果的可信度。例如，如果 $P(y_i = 1 | x_i)$ 接近1，分类器就非常有信心元组 x_i 是一个正例。

我们如何使线性分类器不仅预测元组的类别标签，还能告诉我们它在做出这种预测时的可信度？实现这一目标的有效方法是通过逻辑回归分类器。让我们首先介绍一个重要的函数，称为sigmoid函数，其定义为 $\sigma(z) = 1 / (1 + e^{-(z)}) = e^z / (1 + e^z)$ 。

—— $\frac{1}{1+e^z}$ From Fig. 6.13 我们可以看到，sigmoid函数将一个实数映射到 $(-\infty, +\infty)$ （即图6.13的x轴）映射到输出值范围 $(0, 1)$ （即图6.13的y轴）。因此，如果我们利用sigmoid函数将线性回归模型的输出映射到0和1之间的数字，我们可以将映射结果解释为观察到正类标签的后验概率。这正是逻辑回归分类器所尝试做的！

正式地，我们有 $P(\hat{y}_i = 1 | x_i, w) = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}}$ ，其中 $\hat{y}_i$ 是具有属性 x_i 的元组的预测类别标签， w 是权重向量。注意，我们通过引入一个虚拟属性将偏置项 b 吸收到权重向量 w 中，以简化符号，就像我们在多线性回归模型和感知机中所做的那样。自然地，如果 $P(\hat{y}_i = 1 | x_i, w) > 0.5$ ，分类器预测元组 x_i 是一个正元组（即， $\hat{y}_i = 1$ ），否则，它预测为负元组（即，

$\hat{y}_i = 0$ ）。这（细节留作练习）等价于以下线性分类器：预测 $\hat{y}_i = 1$ （即，正元组）当 $w^T x_i > 0$ 时，预测 $\hat{y}_i = 0$ （即，负元组）当 $w^T x_i < 0$ 。因此如果我们知道权重向量 w ，给定元组的分类任务就相当简单。也就是说，我们

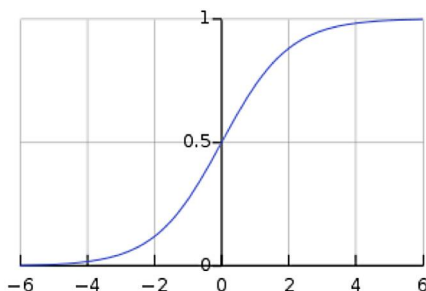


图 6.13

sigmoid 函数的示意图。sigmoid 函数将输入从更大的范围 $(-\infty, +\infty)$ 压缩到一个较小的范围 $(0, 1)$ 。因此，sigmoid 函数也被称为压缩函数。在第 10 章中，我们将看到其他类型的压缩函数，这些在深度学习术语中被称为激活函数。

只需将给定元组的属性向量 x_i 与权重向量 w 相乘，然后根据 $w^T x_i$ 的符号进行预测。如果 $w^T x_i$ 是一个正数，我们预测给定元组是一个正元组。否则，我们预测它是一个负元组。

“我们如何从一组训练元组中确定最佳权重向量 w ？”训练逻辑回归分类器的经典方法（即，从训练集中确定最佳权重向量 w ）是通过最大似然估计（MLE）。再假设有 n 个训练元组 (x_i, y_i) ($i=1, \dots, n$)。由于我们有一个二分类任务，我们可以将预测的类别标签 $\hat{y}_i$ 视为一个伯努利随机变量，它只能取两个可能的值，包括 $P(\hat{y}_i=1|x_i, w)=p_i$ 和 $P(\hat{y}_i=0|x_i, w)=1-p_i$ ，其中 $p_i=\sigma(w^T x_i)=$

确定，它描述了观察到预测类标签（即， $\hat{y}_i=1$ ）为正结果的概率。注意， $\frac{1}{1+e^{-w^T x_i}}$ 由sigmoid函数
标签 y_i 是一个二元变量。因此，我们有 $P(\hat{y}_i=y_i)=p_i^{y_i}(1-p_i)^{1-y_i}$ 。最大似然估计方法旨在解决以下优化问题，即我们应该选择最佳权重向量 w ，以最大化训练集的似然性。直观上，我们希望找到最佳模型参数（即，权重向量 w ），以便观察到整个训练集的“机会”（即，似然性或概率）最高。

$$w^* = \operatorname{argmax}_w L(w) = \prod_{i=1}^n p_i^{y_i} (1-p_i)^{1-y_i} = \prod_{i=1}^n \left(\frac{e^{w^T x_i}}{1+e^{w^T x_i}} \right)^{y_i} \left(\frac{1}{1+e^{w^T x_i}} \right)^{1-y_i} \quad (6.19)$$

“但是，我们如何开发一个算法来解决这个优化问题，以找到最佳权重向量 w ？”首先，我们注意到似然函数 $L(w)$ 有许多非负项相互相乘。在实践中，通常更方便使用这样一个复杂函数的对数。因此，我们有以下等效的优化问题，其中 $l(w)$ 被称为对数似然。

$$w^* = \operatorname{argmax}_w l(w) = \sum_{i=1}^n y_i x_i^T w - \log(1 + e^{w^T x_i}). \quad (6.20)$$

从优化的角度来看，好消息是公式（6.20）中的对数似然函数是一个严格的凹函数，因此其最大值（最优解）是唯一存在的。然而，坏消息是上述优化问题的闭式解并不存在。在这种情况下，一种常见的策略是通过以下方式迭代地寻找最优解 w^* 。在每次迭代中，我们尝试改善当前的权重向量 w ，以便最大化我们希望优化的目标函数（对数似然函数 $l(w)$ ）。为了最大程度地提高当前的目标函数 $l(w)$ ，结果表明，更新当前权重向量 w 的最佳方向是沿着其梯度。由此得出以下算法，从训练集中学习最优权重向量 w^* 。我们从权重向量 w 的初始猜测开始（例如，我们可以简单地设定 $w=0$ ）。然后，学习算法将迭代，直到收敛，或者达到最大迭代次数或其他预设的停止标准。在每次迭代中，它更新权重向量 w 如下： $w \leftarrow w + \eta \sum_{i=1}^n (y_i - P(\hat{y}_i=1|x_i, w))x_i$ ，其中 $\eta > 0$ 是用户指定的学习率。

“那么，上述算法的直觉是什么？”让我们分析每个训练元组 (x_i, y_i) 对更新权重向量 w 的影响。我们考虑两种情况，取决于它是正元组（即， $y_i=1$ ）还是负元组（即， $y_i=0$ ）。对于前者，

给定元组对更新权重向量 w 的影响可以计算为 $w \leftarrow w + \eta(1 - P(\hat{y}_i = 1|x_i, w))x_i$ 。直觉是我们希望将当前权重向量 w 更新到正元组的属性向量 x_i 的方向。对于后者（即， $y_i = 1$ ），给定元组对更新权重向量 w 的影响可以计算为 $w \leftarrow w - \eta P(\hat{y}_i = 1|x_i, w)x_i$ 。直觉是我们希望将当前权重向量 w 更新到远离负元组的属性向量 x_i 的方向。从这个角度来看，训练逻辑回归分类器的学习算法与感知机算法有一些相似之处。也就是说，这两种算法都试图更新当前权重向量 w ，使其（1）更符合正元组的属性向量，并且（2）更不符合（即，朝向负元组的属性向量的相反方向）负元组的属性向量。

然而，这两种算法（感知器与逻辑回归）在更新权重向量 w 的程度上存在差异。在感知器中，它对当前权重向量 w 所有错误预测的元组使用固定的学习率 η 。另一方面，在逻辑回归中，它依赖于学习率 η 以及 $P(\hat{y}_i = 1|x_i, w)$ （即，基于当前权重向量 w ，给定元组属于正类的概率）。这使得逻辑回归算法在以下意义上具有自适应性。例如，如果 $P(\hat{y}_i = 1|x_i, w)$ 对于一个正元组来说很高，这意味着当前权重向量 w 对这个正元组的预测不仅是正确的（即， $P(\hat{y}_i =$

$1|x_i, w) > 0.5$ ），但也相当自信（即， $P(\hat{y}_i = 1|x_i, w)$ 接近1）。那么，这个正元组的影响（即， $\eta(1 - P(\hat{y}_i = 1|x_i, w))$ ）对更新权重向量的影响相对较小。另一方面，如果 $P(\hat{y}_i = 1|x_i, w)$ 对于一个负元组很高，这意味着当前权重向量 w 对这个负元组的预测要么是错误的（即， $P(\hat{y}_i = 1|x_i, w) > 0.5$ ），要么是正确的但置信度低（即， $P(\hat{y}_i = 1|x_i, w)$ 刚好低于0.5）。那么，这个负元组的影响（即， $\eta P(\hat{y}_i = 1|x_i, w)$ ）对更新权重向量的影响将相对较大。换句话说，逻辑回归学习算法更关注那些“困难”的训练元组，这些元组要么被当前权重向量 w 错误预测，要么被正确预测但置信度低。回想一下图6.12(a)中的例子，感知器仅使用 x_1 和 x_8 来更新当前权重向量 w ，因这两个元组被当前 w 错误分类。相比之下，逻辑回归使用所有训练元组来更新权重向量 w 。它们中， x_1 和 x_8 对更新 w 的影响最大，因为它们都被当前分类器错误分类； x_2, x_3, x_5, x_9 和 x_{10} 的影响最小，因为它们都被当前权重向量 w 高度自信地正确分类； x_4, x_6 和 x_7 的影响适中，因为它们被正确分类，但相对置信度较低。

“逻辑回归算法的效果如何？有哪些潜在的局限性以及如何减轻这些局限性？”由于对数似然函数 $l(w)$ 是一个凹函数，上述训练逻辑回归分类器的算法保证收敛到其最优解。然而，如果训练集是线性可分的，算法可能会收敛到一个具有无限大范数的权重向量 w 。（参见图6.14的示例。）一个“大的”权重向量 w 可能使得训练出的分类器容易受到给定元组某些属性的噪声影响。这将反过来导致学习到的逻辑回归分类器的泛化性能较差。换句话说，学习到的逻辑回归分类器过拟合了训练集。减轻过拟合的有效方法是将正则化项 $\|w\|^2$ 引入目标函数 $l(w)$ 中，以防止学习到的权重向量

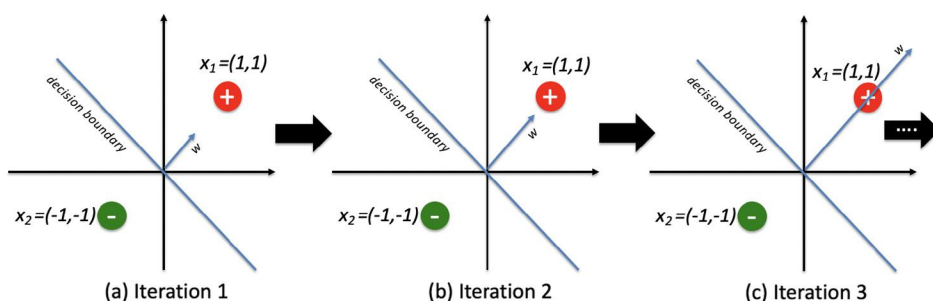


图 6.14

线性可分情况下逻辑回归的无限大权重向量示意图。在二维空间中有两个训练元组，一个正训练元组 $x_1=(1,1)$ 和一个负训练元组 $x_2=(-1,-1)$ 。为简化起见，我们设定偏置标量 $b=0$ 和学习率 $\eta=1$ 。假设在迭代1时，权重向量 $w=(1,1)$ 。然后，上述引入的逻辑回归算法将更新权重向量为 $w_{\text{new}}=w_{\text{old}}+(1-P(\hat{y}_1=1|x_1, w_{\text{old}}))x_1-P(\hat{y}_2=1|x_2, w_{\text{old}})x_2=(0.5,0.5)+a(1,1)=(1+2a)w_{\text{old}}$ ，其中 $a=1-P(\hat{y}_1=1|x_1, w_{\text{old}})+P(\hat{y}_2=1|x_2, w_{\text{old}})>0$ 。因此，新的权重向量 w_{new} 与旧的 w_{old} 方向相同。因此，决策边界保持不变，但新的权重向量 w_{new} 的大小按 $(1+2a)$ 的因子增长。这种趋势将随着逻辑回归算法的进展而持续，导致权重向量的大小无限增大。

变得“过大”。¹¹ 第二个潜在的限制是逻辑回归背后的独立性假设。回想一下，当我们计算训练集的似然性 $L(w)$ 时，我们只是将每个训练元组的似然性相乘（公式 (6.19)）。这意味着我们隐含地假设不同的训练元组彼此独立。然而，这一假设在某些应用中可能会被违反（例如，社交网络上的用户彼此相互连接）。基于图 6.14 的分类可能为这个问题提供一个自然的解决方案。第三个潜在的限制在于计算挑战。注意，在上述更新规则 $w \leftarrow w + \eta \sum_{i=1}^n (y_i - P(\hat{y}_i=1|x_i, w))x_i$ 中，我们需要计算所有训练元组的梯度 $(y_i - P(\hat{y}_i=1|x_i, w))$ ，然后将它们相加以更新权重向量 w 。如果有数百万个训练元组，进行这样的计算在计算上是非常昂贵的。解决这个问题有效方法是使用

随机梯度下降 方法来训练逻辑回归分类器。也就是说，在每次迭代中，我们将随机抽取一小部分训练元组（这通常被称为 **小批量**），并且只使用抽取的元组（而不是所有训练元组）来更新权重向量。值得指出的是，随机梯度下降在许多其他数据挖掘算法中被广泛使用，例如深度学习方法，这将在第10章中介绍。

¹¹ 从统计参数估计的角度来看，我们正在从最大似然估计（MLE）转向最大后验估计（MAP）。在 $l(w)$ 中添加正则化项 $\|w\|^2$ 相当于对权重向量 w 施加均值向量为原点的高斯先验。

6.6 模型评估与选择

现在您可能已经构建了一个分类模型，心中可能有许多问题。例如，假设您使用了之前销售的数据来构建一个分类器，以预测客户的购买行为。您希望估计分类器在预测未来客户的购买行为方面的准确性，即分类器未经过训练的未来客户数据。您甚至可能尝试了不同的方法来构建多个分类器，现在希望比较它们的准确性。但是，什么是准确性？我们如何估计它？分类器准确性的某些度量是否比其他度量更合适？我们如何获得可靠的准确性估计？

这些问题将在本节中讨论。

第6.6.1节描述了分类器预测准确性的各种评估指标。基于对给定数据的随机抽样分区，保留和随机子抽样（第6.6.2节）、交叉验证（第6.6.3节）和自助法（第6.6.4节）是评估准确性的常用技术。如果我们有多个分类器并希望选择“最佳”一个，这被称为模型选择（即选择一个分类器而不是另一个）。最后两节讨论了这个问题。第6.6.5节讨论如何使用统计显著性检验来评估两个分类器之间的准确性差异是否由于偶然因素造成。第6.6.6节介绍了如何基于成本效益和接收者操作特征（ROC）曲线比较分类器。

6.6.1 评估分类器性能的指标

本节介绍了评估分类器在预测元组类别标签时的好坏或“准确性”的度量。我们将考虑类别元组分布较为均匀的情况，以及类别不平衡的情况（例如，在医学测试中，某些重要类别可能很少见）。本节中介绍的分器评估指标总结在图 6.15 中。它们包括准确率（也称为识别率）、灵敏度（或召回率）、特异性、精确率、F1 和 $F\beta$ 。请注意，尽管准确率是一个特定的度量，但“准确性”一词也被用作一般术语，指代分类器的预测能力。

使用训练数据来推导分类器，然后估计学习模型的准确性可能会由于学习算法对数据的过度专业化而导致误导性的过于乐观的估计。（我们稍后会对此进行更多讨论！）相反，最好在一个测试集上测量分类器的准确性，该测试集由未用于训练模型的类别标记元组组成。

在讨论各种度量之前，我们需要熟悉一些术语。

请记住，我们可以用正元组（主要关注类的元组）和负元组（所有其他元组）来进行讨论。给定两个类别，例如，正元组可能是 `buys_computer = yes`，而负元组是 `buys_computer = no`。假设我们有一个标记元组的测试集上使用我们的分类器。P 是正元组的数量，N 是负元组的数量。对于每个元组，我们将分类器的类别标签预测与元组的已知类别标签进行比较。

我们需要了解四个额外的术语，它们是计算各种评估指标时使用的“构建块”。理解它们将使我们更容易掌握各种指标的含义。

¹² 在机器学习和模式识别文献中，这些分别被称为正样本和负样本。分别。

指标	公式
准确率, 识别率	$\frac{TP+TN}{P+N}$
错误率, 误分类率	$\frac{FP+FN}{P+N}$
灵敏度, 真正阳性率, 召回率	$\frac{TP}{P}$
特异性, 真正阴性率	$\frac{TN}{N}$
精确度	$\frac{TP}{TP+FP}$
F_1, F -分数, 精确度和召回率的调和平均数	$\frac{2 \times \text{精确度} \times \text{召回率}}{\text{精确度} + \text{召回率}}$
F_β , 其中 β 是一个非负实数	$\frac{(1+\beta^2) \times \text{精确度} \times \text{召回率}}{\beta^2 \times \text{精确度} + \text{召回率}}$

图 6.15

评估指标。请注意, 有些指标有多个名称。TP, TN, FP, FN, P, N 分别指代真正例、真实负例、假正例、假负例、正样本和负样本的数量 (见正文)。

		预测类别		
实际类别 _{yes}		<i>yes</i>	<i>no</i>	总计
	否	<i>TP</i>	<i>FN</i>	<i>P</i>
		<i>FP</i>	<i>TN</i>	<i>N</i>
	总计	<i>P'</i>	<i>N'</i>	<i>P+N</i>

图 6.16

混淆矩阵, 显示正负元组的总数。

- 真正例 (TP): 这些是被分类器正确标记的正元组。
设 TP 为真正例的数量。
- 真负例 (TN): 这些是被分类器正确标记的负元组。设
TN 为真负例的数量。
- 假正例 (FP): 这些是被错误标记为正的正元组 (例如,
类元组 `buys_computer=no`, 但分类器预测 `buys_computer=yes`)。设
FP 为假正例的数量。
- 假负例 (FN): 这些是被错误标记为负的正元组 (例如, 类元组 `buys_computer=yes`, 但分
类器预测 `buys_computer=no`)。设 FN 为假负例的数量。

这些术语在图 6.16 的混淆矩阵中进行了总结。

混淆矩阵是分析分类器识别不同类别元组能力的有用工具。TP 和 TN 告诉我们分类器何时正确, 而 FP 和 FN 则告诉我们分类器何时出错 (即错误标记)。给定 m 类 (其中 $m \geq 2$), 混淆矩阵是一个至少为 $m \times m$ 的表格。第 i 行第 j 列的条目 $CM(i, j)$ 表示被分类器标记为类别 j 的类别 i 的元组数量。为了使分类器具有良好的准确性, 理想情况下, 大多数元组应沿着混淆矩阵的对角线表示, 从条目 $CM(1, 1)$ 到条目 $CM(m, m)$, 其余条目应为零或接近零。也就是说, 理想情况下, FP 和 FN 应接近零。

类别	购买_电脑= yes	购买_电脑= no	总计	识别率 (%)
<i>buys_computer= yes</i>	6954	46	7000	99.34
<i>buys_computer= no</i>	412	2588	3000	86.27
总计	7366	2634	10,000	95.42

图 6.17

类别的混淆矩阵 *buys_computer= yes* 和 *buys_computer= no*，其中第 i 行和第列的条目 um_{ij} 显示了被分类器标记为类 j 的类 i 的元组数量。理想情况下，非对角线条目应为零或接近零。

该表可能有额外的行或列以提供总计。例如，在图 6.16 的混淆矩阵中，显示了 P 和 N 。此外， P' 是被标记为正类的元组数量 ($TP + FP$)，而 N' 是被标记为负类的元组数量 ($TN + FN$)。

元组的总数为 $TP + TN + FP + FN$ ，或 $P + N$ ，或 $P' + N'$ 。请注意，尽管所示的混淆矩阵是针对二分类问题，但混淆矩阵也可以以类似方式轻松绘制多类。

现在让我们来看一下评估指标，从准确率开始。分类器在给定测试集上的准确率是被分类器正确分类的测试集元组的百分比。也就是说，

$$accuracy = \frac{TP + TN}{P + N}. \quad (6.21)$$

在模式识别文献中，这也被称为分类器的整体识别率；也就是说，它反映了分类器识别各种类元组的能力。图 6.17 给出了两个类 *buys_computer= yes* (正类) 和 *buys_computer= no* (负类) 的混淆矩阵示例。显示了总计，以及每个类和整体的识别率。通过查看混淆矩阵，可以很容易地看出相应的分类器是否将两个类混淆。

例如，我们看到它错误地将 412 “no” 元组标记为 “yes。” 当类别分布相对平衡时，准确性最为有效。

我们还可以谈论分类器的错误率或误分类率 M ，简单来说就是 $1 - accuracy(M)$ ，其中 $accuracy(M)$ 是 M 的准确性。这也可以计算为

$$error\ rate = \frac{FP + FN}{P + N}. \quad (6.22)$$

如果我们使用训练集（而不是测试集）来估计模型的错误率，这个量被称为重新替代错误。¹³ 这个错误估计对真实错误率是乐观的（同样，相应的准确性估计也是乐观的），因为模型没有在任何它未见过的样本上进行测试。

我们现在考虑类别不平衡问题，主要关注的类别是稀有的。也就是说，数据集分布反映了负类的显著多数和正类的少数。例如，在欺诈检测应用中，关注的类别（或正类）是“欺诈”其发生频率远低于负类“非欺诈”类别。在医学数据中，可能

¹³ 在机器学习文献中，通常称其为训练错误。

类别	是	否	总计	识别率 (%)
是	90	210	300	30.00
否	140	9560	9700	98.56
总计	230	9770	10,000	96.40

图 6.18

癌症类别的混淆矩阵 癌症= 是 和 癌症= 否。

可能是一个稀有类别，例如“癌症。”假设你已经训练了一个分类器来对医疗数据元组进行分类，其中类别标签属性是“癌症”，可能的类别值是“是”和“否。”比如，97%的准确率可能让分类器看起来相当准确，但如果只有大约3%的训练元组实际上是癌症呢？显然，97%的准确率可能是不可接受的——例如，分类器可能只正确标记了非癌症元组，而错误分类了所有癌症元组。相反，我们需要其他指标来评估分类器识别正元组（癌症= 是）和识别负元组（癌症= 否）的能力。

敏感性和特异性指标可以分别用于此目的。敏感性也称为真阳性(识别)率（即，正确识别的正元组的比例），而特异性是真阴性率（即，正确识别的负元组的比例）。这些指标的定义如下

$$sensitivity = \frac{TP}{P} \quad (6.23)$$

$$specificity = \frac{TN}{N}. \quad (6.24)$$

可以证明，准确性是敏感性和特异性的函数：

$$accuracy = sensitivity \frac{P}{(P+N)} + specificity \frac{N}{(P+N)}. \quad (6.25)$$

示例 6.9. 敏感性和特异性。图6.18 显示了医疗数据的混淆矩阵

其中类值为 是 和 否，类标签属性为 癌症。分类器的敏感性为

$\frac{90}{300} = 30.00\%$ 。特异性为 $\frac{9560}{9700} = 98.56\%$ 。分类器的整体准确性为 $\frac{9650}{10000} = 96.50\%$ 。因此，我们注意到，尽管分类器具有高准确性，但由于其低敏感性，正确标记正类（稀有类）的能力较差。它具有高特异性，这意味着它可以准确识别负元组。处理类别不平衡数据的技术在第6.7.5 节中给出。 □

精确度和召回率 衡量在分类中也被广泛使用。精确度可以被视为精确性的度量（即，被标记为正的元组中实际为正的百分比），而召回率是完整性的度量（被标记为正的元组中实际为正的百分比）。如果召回率听起来很熟悉，那是因为它与敏感性（或真正率）是相同的。这些度量可以计算为

$$precision = \frac{TP}{TP + FP} \quad (6.26)$$

$$recall = \frac{TP}{TP + FN} = \frac{TP}{P}. \quad (6.27)$$

示例 6.10. 精确率和召回率。图 6.18 中分类器对 *yes* 类的精确率为 $\frac{9}{23} = 39.13\%$ 。召回率为 $\frac{90}{300} = 30.00\%$ ，这与示例 6.9 中的敏感性计算相同。

9
2
3
0

□

对于类 *C*，完美的精确率得分为 1.0 意味着分类器标记为属于类 *C* 的每个元组确实属于类 *C*。然而，这并没有告诉我们分类器错误标记的类 *C* 元组的数量。对于 *C*，完美的召回率得分为 1.0 意味着类 *C* 中的每个项目都被标记为如此，但它并没有告诉我们有多少其他元组被错误标记为属于类 *C*。精确率和召回率之间往往存在反向关系，增加一个可能会以降低另一个为代价。例如，我们的医疗分类器可能通过将所有以某种方式呈现的癌症元组标记为 *cancer* 来实现高精确率，但如果它错误标记了许多其他癌症元组，则可能会导致低召回率。精确率和召回率得分通常是一起使用的，其中在固定的召回值下比较精确率值，或反之亦然。例如，我们可以在召回值为 0.75 时比较精确率值。

使用精确率和召回率的另一种方法是将它们结合成一个单一的度量。这就是 *F* 度量（也称为 *F*₁ 分数或 *F*-分数）和 *F*_β 度量的方法。它们的定义为

$$F = \frac{2 \times precision \times recall}{precision + recall} \quad (6.28)$$

$$F_{\beta} = \frac{(1 + \beta^2) \times precision \times recall}{\beta^2 \times precision + recall}, \quad (6.29)$$

其中 β 是一个非负实数。*F* 度量是精确率和召回率的调和平均数（其证明留作练习）。它对精确率和召回率赋予相等的权重。*F*_β 度量是精确率和召回率的加权度量。它将 β 倍的权重分配给召回率，而不是精确率。

常用的 *F*_β 度量有 *F*₂（它将召回率的权重加倍于精确率）和 *F*_{0.5}（它将精确率的权重加倍于召回率）。

“还有其他情况下准确率可能不合适吗？”在分类问题中，通常假设所有元组都是唯一可分类的，也就是说，每个训练元组只能属于一个类。然而，由于大型数据库中数据的广泛多样性，假设所有元组都是唯一可分类的并不总是合理的。相反，更可能的假设是每个元组可能属于多个类。那么，如何测量大型数据库中分类器的准确性呢？

准确性度量不合适，因为它没有考虑元组属于多个类别的可能性。

与其返回类别标签，不如返回类别概率分布。准确性度量可以使用一种“第二次猜测”启发式方法，即如果类别预测与第一或第二个最可能的类别一致，则判断为正确。尽管这在某种程度上考虑了元组的非唯一分类，但这并不是一个完整的解决方案。

除了基于准确性的度量外，分类器还可以在以下附加方面进行比较：

- 速度：这指的是生成和使用给定分类器所涉及的计算成本。

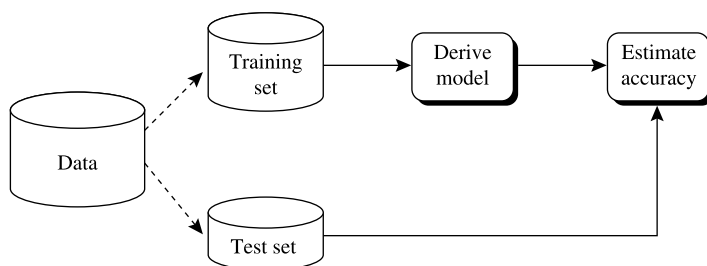


图6.19

使用保留法估计准确性。

- 鲁棒性：这是分类器在给定噪声数据或缺失值数据时做出正确预测的能力。鲁棒性通常通过一系列代表不同噪声和缺失值程度的合成数据集进行评估。
- 可扩展性：这指的是在给定大量数据时有效构建分类器的能力。可扩展性通常通过一系列逐渐增大的数据集进行评估。
- 可解释性：这指的是分类器或预测器提供的理解和洞察的水平。可解释性可能是主观的，因此更难以评估。决策树和分类规则通常易于解释，但随着它们变得复杂，其可解释性可能会降低。我们将在第7章介绍一些基本技术，以提高分类模型的可解释性。

总之，我们介绍了几种评估指标。当数据类别相对均匀分布时，准确度指标效果最佳。其他指标，如灵敏度（或召回率）、特异性、精确度、F 和 F_β ，更适合处理类别不平衡问题，其中主要关注的类别较为稀少。其余小节将重点讨论如何获得可靠的分类器准确度估计。

6.6.2 保留法和随机子抽样

保留法是在我们讨论准确度时提到的内容。在这种方法中，给定的数据被随机划分为两个独立的集合，一个是训练集，另一个是测试集。通常，三分之二的分配给训练集，剩下的三分之一分配给测试集。训练集用于推导模型。然后使用测试集来估计模型的准确度（图6.19）。这个估计是悲观的，因为只有一部分初始数据用于推导模型。

随机子抽样 是一种保留法的变体，其中保留法重复进行 k 次。整体准确性估计被视为每次

迭代获得的准确性的平均值。

6.6.3 交叉验证

在 k 折交叉验证中，初始数据被随机划分为 k 个互斥的子集或“折” $D_1, D_2, \dots, D_k$ ，每个子集的大小大致相等。训练和测试在 k

在第 i 次迭代中，分区 D_i 被保留为测试集，其余分区则共同用于训练模型。也就是说，在第一次迭代中，子集 $D_2, \dots, D_k$ 共同作为训练集以获得第一个模型，该模型在 D_1 上进行测试；第二次迭代则在子集上进行训练 $D_1, D_3, \dots, D_k$ 并在 D_2 上进行测试；依此类推。与保留法和随机子抽样方法不同，这里每个样本用于训练的次数相同，并且用于测试一次。对于分类，准确性估计是 k 次迭代中正确分类的总数，除以初始数据中的元组总数。

留一交叉验证是 k 折交叉验证的一个特例，其中 k 被设置为初始元组的数量。也就是说，每次仅“留出”一个样本作为测试集。留一交叉验证通常在初始数据集较小时使用。在分层交叉验证中，各折是分层的，以便每个折中元组的类别分布大致与初始数据中的类别分布相同。

在实践中，推荐使用分层10折交叉验证来估计准确性（即使计算能力允许使用更多折），因为它的偏差和方差相对较低。

6.6.4 自助法

与刚提到的准确性估计方法不同，自助法（bootstrap method）以有放回的方式均匀抽样给定的训练元组。也就是说，每次选择一个元组时，它被再次选择并重新添加到训练集的可能性是相等的。例如，想象一台随机选择元组作为我们训练集的机器。在有放回的抽样中，这台机器可以多次选择同一个元组。

有几种自助法。常用的一种是.632自助法，其工作原理如下。假设我们有一个包含 d 个元组的数据集。该数据集被抽样 d 次，采用有放回的方式，结果形成一个包含 d 个样本的自助样本或训练集。原始数据元组中的一些可能在这个样本中出现多次。未进入训练集的数据元组最终形成测试集。假设我们多次尝试这一过程。结果表明，平均而言，63.2%的原始数据元组将出现在自助样本中，其余的36.8%将形成测试集（因此得名.632自助法）。

“这个数字63.2%是从哪里来的？”每个元组被选择的概率为 $1/d$ ，因此不被选择的概率为 $(1-1/d)$ 。我们需要选择 d 次，因此在整个过程中一个元组不被选择的概率为 $(1-1/d)^d$ 。如果 d 很大，这个概率接近 $e^{-1}=0.368$ 。¹⁴因此36.8%的元组将不会被选入训练集，从而最终进入测试集，剩下的63.2%将形成训练集。

我们可以重复抽样程序 k 次，在每次迭代中，我们使用当前的测试集来获得从当前自助样本中获得的模型的估计准确性。模型的整体准确性 M 然后估计为

$$Acc(M) = \frac{1}{k} \sum_{i=1}^k (0.632 \times Acc(M_i)_{test_set} + 0.368 \times Acc(M_i)_{train_set}), \quad (6.30)$$

¹⁴ e 是自然对数的底数，即 $e=2.718$ 。

其中 $Acc(M_i)_{test_set}$ 是使用自助样本 i 获得的模型在应用于测试集 i 时的准确性。 $Acc(M_i)_{train_set}$ 是使用自助样本 i 获得的模型在应用于原始数据元组集时的准确性。自助法往往过于乐观。它在小数据集上效果最佳。

6.6.5 使用统计显著性检验进行模型选择

假设我们从数据中生成了两个分类模型， M_1 和 M_2 。我们进行了10折交叉验证，以获得每个模型的平均错误率¹⁵。我们如何确定哪个模型更好？选择错误率最低的模型似乎是直观的；然而，平均错误率只是对未来数据案例真实总体错误的估计。在任何给定的10折交叉验证实验中，错误率之间可能存在相当大的方差。尽管 M_1 和 M_2 的平均错误率看起来不同，但这种差异可能并不具有统计显著性。如果这两者之间的任何差异仅仅是偶然造成的呢？本节将讨论这些问题。

为了确定两个模型的平均错误率是否存在任何“真实”的差异，我们需要使用统计显著性检验。此外，我们希望为我们的平均错误率获得一些置信区间，以便我们可以做出类似于“任何观察到的平均值在未来样本中不会变化±两个标准误差的95%时间”或“一个模型比另一个模型的误差范围更好±4%”的陈述。

我们需要什么来进行统计检验？假设对于每个模型，我们进行了10折交叉验证，比如说，进行了10次，每次使用不同的10折数据划分。每个划分都是独立抽取的。我们可以分别对模型 M_1 和 M_2 获得的10个误差率进行平均，以获得每个模型的平均误差率。对于给定的模型，在交叉验证中计算的个别误差率可以视为来自概率分布的不同独立样本。

一般来说，它们遵循具有 $k-1$ 自由度的 t 分布，其中，这里， $k=10$ 。（这个分布看起来与正态分布或高斯分布非常相似，尽管定义这两者的函数是相当不同的。两者都是单峰的、对称的，并且呈钟形。）这使我们能够进行假设检验，其中使用的显著性检验是 t 检验或学生 t 检验。我们的假设是这两个模型是相同的，换句话说，它们之间的平均误差率差异为零。如果我们能够拒绝这个假设（称为零假设），那么我们可以得出结论，两个模型之间的差异在统计上是显著的，在这种情况下，我们可以选择误差率较低的模型。

在数据挖掘实践中，我们可能经常使用一个单一的测试集，也就是说，同一个测试集可以同时用于 M_1 和 M_2 。在这种情况下，我们对两个模型进行逐对比较，每个10折交叉验证轮次都如此。也就是说，对于第 i 轮10折交叉验证，使用相同的交叉验证划分来获得 M_1 和 M_2 的错误率。设 $err(M_1)_i$ （或 $err(M_2)_i$ ）为模型 M_1 （或 M_2 ）在第 i 轮的错误率。 M_1 的错误率取平均值以获得 M_1 的平均错误率，记作 $err(M_1)$ 。同样，我们可以获得 $err(M_2)$ 。两个模型之间差异的方差记作 $var(M_1 - M_2)$ 。 t 检验计算具有 $k-1$ 自由度的 t 统计量， k 为样本数。在我们的例子中， $k=10$ ，因为这里的 k 样本是我们获得的错误率。

¹⁵ 请记住，模型 M 的错误率为 $1 - accuracy(M)$ 。

从每个模型的十次 10 折交叉验证中计算得出。逐对比较的 t 统计量计算如下：

$$t = \frac{\overline{err}(M_1) - \overline{err}(M_2)}{\sqrt{var(M_1 - M_2)/k}}, \quad (6.31)$$

其中

$$var(M_1 - M_2) = \frac{1}{k} \sum_{i=1}^k [err(M_1)_i - err(M_2)_i - (\overline{err}(M_1) - \overline{err}(M_2))]^2. \quad (6.32)$$

为了确定 M_1 和 M_2 是否显著不同，我们计算 t 并选择一个显著性水平 sig 。在实践中，通常使用 5% 或 1% 的显著性水平。然后我们查阅统计学标准教材中提供的 t 分布表。该表通常按自由度作为行，显著性水平作为列进行排列。假设我们想确定 M_1 和 M_2 之间的差异在 95% 的总体中是否显著，即 $\text{sig} = 5\%$ 或 0.05。我们需要从表中找到对应于 $k - 1$ 自由度（或我们例子中的 9 自由度）的 t 分布值。然而，由于 t 分布是对称的，通常只显示分布的上百分位点。因此，我们查找 $z = \text{sig}/2$ 的表值，在这种情况下为 0.025，其中 z 也称为置信限。如果 $t > z$ 或 $t < -z$ ，则我们的 t 值位于拒绝区域，即分布的尾部。这意味着我们可以拒绝零假设，即 M_1 和 M_2 的均值相同，并得出结论，两个模型之间存在统计显著差异。否则，如果我们无法拒绝零假设，我们得出的结论是 M_1 和 M_2 之间的任何差异都可以归因于偶然。

如果有两个测试集而不是单个测试集，则使用非配对版本的 t 检验，其中两个模型均值之间的方差估计为

$$var(M_1 - M_2) = \frac{var(M_1)}{k_1} + \frac{var(M_2)}{k_2}, \quad (6.33)$$

且 k_1 和 k_2 是用于 M_1 和 M_2 的交叉验证样本数量（在我们的案例中为 10 折交叉验证轮次）。这也被称为两个样本 t 检验。当查阅 t 分布表时，使用的自由度数量取为两个模型的最小自由度。

6.6.6 基于成本效益和 ROC 曲线比较分类器

真正例、真负例、假正例和假负例在评估与分类模型相关的成本和收益（或风险和收益）时也很有用。假负例（例如错误地预测癌症患者为非癌症患者）所涉及的成本远大于假正例（错误但保守地将非癌症患者标记为癌症患者）。在这种情况下，我们可以通过为每种错误分配不同的成本来权衡一种错误相对于另一种错误。这些成本可能考虑到对患者的危险、由此产生的治疗的财务成本以及其他医院成本。同样，真正例决策所带来的收益可能与真负例的收益不同。到目前为止，为了计算分类器的准确性，我们假设成本相等，并基本上将真正例和真负例的总和除以测试元组的总数。

另外，我们可以通过计算每个决策的平均成本（或收益）来综合考虑成本和收益。其他涉及成本-收益分析的应用包括贷款申请决策和目标营销邮件投放。例如，向违约者贷款的成本远远超过拒绝向非违约者贷款所造成的业务损失。同样，在一个试图识别可能对某些促销材料邮件投放做出响应的家庭的应用中，向许多不响应的家庭投放邮件的成本可能超过不向那些会响应的家庭投放邮件所造成的业务损失。在整体分析中，还需考虑收集数据和开发分类工具的成本。

接收者操作特征曲线是比较两个分类模型的有用视觉工具。ROC曲线源于信号检测理论，该理论是在第二次世界大战期间为分析雷达图像而开发的。给定模型的ROC曲线显示了真正阳性率（TPR）和假阳性率（FPR）之间的权衡。给定测试集和模型，TPR是模型正确标记的正（或“是”）元组的比例；FPR是被错误标记为正的负（或“否”）元组的比例。回想一下，TP、FP、P和N分别是真正阳性、假阳性、正和负元组的数量。从第6.6.1节中，我们知道 $TPR = \frac{TP}{P}$ ，即灵敏度。此外， $FPR = \frac{FP}{N}$ ，即1-特异性。

对于一个二分类问题，ROC曲线使我们能够可视化模型在不同测试集部分中准确识别正例的比率与错误将负例识别为正例的比率之间的权衡。任何TPR的增加都以FPR的增加为代价。ROC曲线下的面积是模型准确性的一个衡量标准。

要为给定的分类模型M绘制ROC曲线，该模型必须能够返回每个测试元组的预测类别概率。通过这些信息，我们对元组进行排名和排序，使得最有可能属于正类或“是”类的元组出现在列表的顶部，而最不可能属于正类的元组则位于列表的底部。朴素贝叶斯（第6.3节）和逻辑回归（第6.5节）分类器为每个预测返回类别概率分布，因此是合适的，尽管其他分类器，如决策树分类器（第6.2节），也可以轻松修改以返回类别概率预测。设一个概率分类器为给定元组X返回的值为 $f(X) \rightarrow [0, 1]$ 。对于二元问题，通常选择一个阈值t，使得 $f(X) \geq t$ 的元组被视为正类，所有其他元组被视为负类。请注意，真正例和假正例的数量都是t的函数，因此我们可以写成TP(t)和FP(t)。两者都是单调非增函数。

我们首先描述绘制ROC曲线的基本思路，然后跟随一个示例。ROC曲线的纵轴表示TPR。横轴表示FPR。对于M的ROC曲线，我们如下开始。从左下角（其中 $TPR = FPR = 0$ ）开始，我们检查列表顶部元组的实际类别标签。如果我们有一个真正的正例（即，正确分类的正元组），那么TP因此TPR增加。在图上，我们向上移动并绘制一个点。如果相反，模型将一个负元组分类为正，则我们有一个假正例，因此两者

FP和FPR增加。在图上，我们向右移动并绘制一个点。这个过程对每个按排名顺序的测试元组重复，每次对于真正的正例在图上向上移动，或者对于假正例向右移动。

元组 #	类别	概率	TP	FP	TN	FN	TPR	FPR	
1	P	0.90	1	0	5	4	0.2	0	
2	P	0.80	2	0	5	3	0.4	0	
3	N	0.70	2	1	4	3	0.4	0.2	
4	P	0.60	3	1	4	2	0.6	0.2	
5	P	0.55	4	1	4	1	0.8	0.2	
6	N	0.54	4	2	3	1	0.8	0.4	
7	N	0.53	4	3	2	1	0.8	0.6	
8	N	0.51	4	4	1	1	0.8	0.8	
9	P	0.50	5	4	1	0	1.0	0.8	
10	N	0.40	5	5	0	0	1.0	1.0	

图 6.20

按得分降序排列的元组，其中得分是概率分类器返回的值。

示例 6.11. 绘制 ROC 曲线。图 6.20 显示了概率分类器为测试集中每个元组返回的概率值（第 3 列），按概率降序排列。

第 1 列仅是元组识别编号，有助于我们的解释。第 2 列是元组的实际类别标签。共有五个正元组和五个负元组；因此 $P=5$ 和 $N=5$ 。随着我们检查每个元组的已知类别标签，我们可以确定其余列的值，

TP, FP, TN, FN, TPR 和 FPR 。我们从元组 1 开始，它具有最高的概率得分，并将该得分作为我们的阈值，即 $t=0.9$ 。因此，分类器将元组 1 视为正类，所有其他元组都被视为负类。由于元组 1 的实际类别标签为正类，我们得到了一个真正的正类，因此 $TP=1$ 和 $FP=0$ 。在剩下的九个元组中，所有都被分类为负类，其中五个实际上是负类（因此， $TN=5$ ）。其余四个实际上都是正类；因此，

$FN=4$ 。因此我们可以计算 $TPR = \frac{TP}{P} = \frac{1}{5} = 0.2$ ，而 $FPR = 0$ 。因此我们有点 $(0.2, 0)$ 用于 ROC 曲线。

接下来，阈值 t 设置为 0.8，这是元组 2 的概率值，因此这个元组现在也被视为正类，而元组 3 到 10 被视为负类。元组 2 的实际类别标签是正类，因此现在 $TP=2$ 。该行的其余部分可以很容易地计算，结果为点 $(0.4, 0)$ 。

接下来，我们检查元组 3 的类别标签，并让 t 为 0.7，这是分类器为该元组返回的概率值。因此元组 3 被视为正类，但其实际标签是负类，因此它是一个假阳性。因此 TP 保持不变， FP 增加，使得 $FP=1$ 。该行中的其余值也可以很容易地计算，得到点 $(0.4, 0.2)$ 。通过检查每个元组得到的 ROC 图是图 6.21 中所示的锯齿线。

有许多方法可以从这些点中获得曲线，其中最常见的是使用一个凸包。该图还显示了一条对角线，对于该模型的每个真正阳性，我们同样可能遇到一个假阳性。作为比较，这条线代表随机猜测。 □

图 6.22 显示了两个分类模型的 ROC 曲线。表示随机猜测的对角线也被显示出来。因此，模型的 ROC 曲线越接近对角线，模型的准确性就越低。如果模型真的很好，最初我们在排名列表中向下移动时更有可能遇到真正的正例。因此，曲线从零开始急剧上升。后来，随着我们开始遇到越来越少的真正正例和越来越多的假正例，曲线逐渐平缓，变得更加水平。

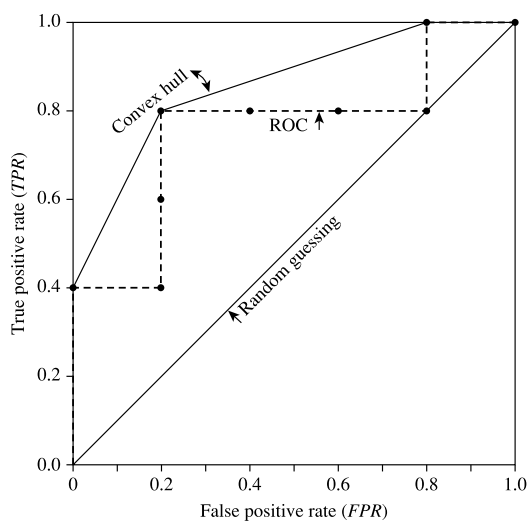


图 6.21

图 6.20 中数据的 ROC 曲线。

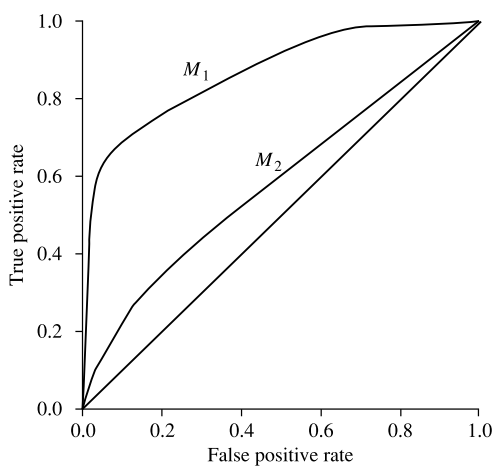


图 6.22

两个分类模型的 ROC 曲线， M_1 和 M_2 。对角线显示了对于每个真正正例，我们同样可能遇到一个假正例。ROC 曲线越接近对角线，模型的准确性就越低。因此， M_1 在这里更准确。

为了评估模型的准确性，我们可以测量曲线下面积（AUC）。几个软件包能够执行这样的计算。面积越接近 0.5，相关模型的准确性就越低。一个具有完美准确性的模型将具有 1.0 的 AUC。

6.7 提高分类准确性的技术

在本节中，您将学习一些提高分类准确性的技巧。我们专注于 **集成**

方法。集成分类是一个复合模型，由多个分类器的组合构成。

各个分类器进行投票，集成根据投票结果返回一个类别标签预测。集成通常比其组成的分类器更准确。我们在第6.7.1节开始介绍集成方法。袋装法（第6.7.2节）、提升法（第6.7.3节）和随机森林（第6.7.4节）是流行的集成方法。

传统学习模型假设数据类别分布良好。然而，在许多现实世界的的数据领域中，数据存在类别不平衡的情况，主要关注的类别仅由少数元组表示。这被称为 **类别不平衡问题**。我们还研究了提高类别不平衡数据分类准确性的技术。这些内容在第6.7.5节中介绍。

6.7.1 介绍集成方法

袋装法、提升法和随机森林是集成方法的例子（图6.23）。集成结合了一系列 k 个学习模型（或基分类器）， $M_1, M_2, \dots, M_k$ ，旨在创建一个改进的复合分类模型， M^* 。给定数据集 D ，用于创建 k 个训练集， $D_1, D_2, \dots, D_k$ ，其中 D_i ($1 \leq i \leq k$) 用于生成分类器 M_i 。给定一个新的数据元组进行分类，各基分类器通过返回类别预测进行投票。集成根据基分类器的投票结果返回类别预测。

一个集成模型的准确性往往高于其基础分类器。例如，考虑一个进行多数投票的集成模型。也就是说，给定一个要分类的元组 X ，它收集基础分类器返回的类别标签预测，并输出多数类别。基础分类器可能会出错，但只有当超过一半的基础分类器出错时，集成模型才会错误地分类 X 。

当模型之间存在显著多样性时，集成模型的效果更好。也就是说，理想情况下，

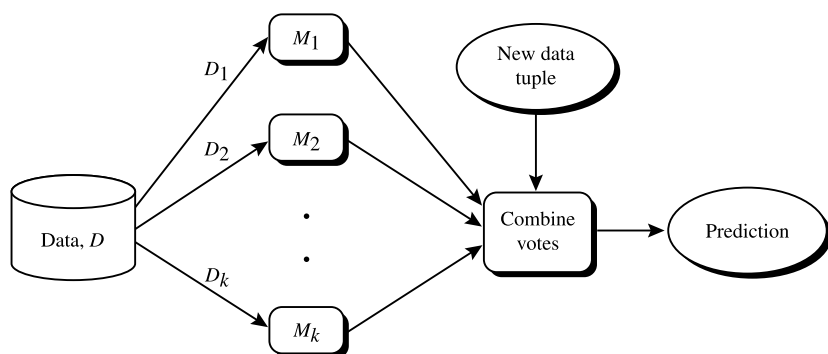


图 6.23

提高分类器准确性。集成方法生成一组分类模型， $M_1, M_2, \dots, M_k$ 。给定一个新的数据元组进行分类，每个分类器为该元组的类别标签“投票”。集成模型结合这些投票以返回类别预测。

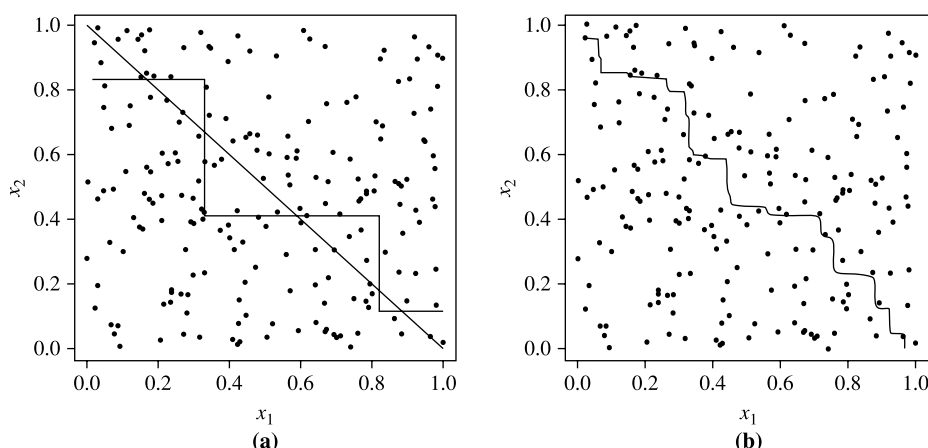


图 6.24

由 (a) 单个决策树和 (b) 决策树集成模型形成的决策边界，用于线性可分问题（即，实际决策边界是一条直线）。决策树在逼近线性边界时遇到困难。集成模型的决策边界更接近真实边界。来源：来自 Seni 和 Elder [SE10]。© 2010 Morgan & Claypool Publishers；经许可使用。

分类器之间的相关性很小。基本分类器的表现也应该优于随机猜测。每个基本分类器可以分配给不同的 CPU，因此集成方法是可并行化的。

为了帮助说明集成的强大，考虑一个由两个属性描述的简单二分类问题， x_1 和 x_2 。该问题具有线性决策边界。图 6.24(a) 显示了决策树分类器在该问题上的决策边界。图 6.24(b) 显示了同一问题上决策树分类器集成的决策边界。尽管集成的决策边界仍然是分段常数的，但它具有更细的分辨率，并且优于单棵树的决策边界。

6.7.2 自助法

我们现在直观地看一下自助法如何作为提高准确性的一种方法。假设你是一位患者，希望根据你的症状进行诊断。你可以选择询问几位医生，而不是只问一位。如果某种诊断的出现频率高于其他诊断，你可以选择这个作为最终或最佳诊断。也就是说，最终诊断是基于多数投票做出的，每位医生的投票权是相等的。现在将每位医生替换为一个分类器，这就是自助法背后的基本思想。直观上，由一大群医生做出的多数投票可能比由一小群医生做出的多数投票更可靠。

给定一个集合， D ，包含 d 个元组，**bagging** 的工作方式如下。对于迭代 i ($i = 1, 2, \dots, k$)，从原始元组集合 D 中有放回地抽取一个训练集， D_i ，包含 d 个元组。请注意，术语 *bagging* 代表 *bootstrap aggregation*。每个训练集都是一个自助样本，如第 6.6.4 节所述。由于使用了有放回的抽样，原始集合 D 中的一些元组可能不会被抽取。

算法：Bagging。Bagging算法——为学习方案创建一个分类模型的集合，其中每个模型给出同等权重的预测。

输入：

- D ，包含 d 个训练元组的集合；
- k ，集合中模型的数量；
- 一个分类学习方案（例如，决策树算法、朴素贝叶斯等）。

输出：集合——一个复合模型， M^* 。

方法：

- (1) 对于 $i=1$ 到 k 执行 // 创建 k 个模型：
- (2) 通过有放回地抽样 D 创建自助样本， D_i ；
- (3) 使用 D_i 和学习方案来推导模型， M_i 。
- (4) **endfor**

要使用集成来分类一个元组， X ：

 让每个 k 模型对 X 进行分类并返回多数投票；

图 6.25

袋装法。

被包含在 D_i 中，而其他模型可能出现多次。为每个训练集 D_i 学习一个分类器模型， M_i 。要对一个未知元组 X 进行分类，每个分类器 M_i 返回其类别预测，这算作一票。袋装分类器 M^* 统计投票并将获得最多票数的类别分配给 X 。袋装法可以通过对给定测试元组的每个预测取平均值来应用于连续值的预测。该算法在图6.25中进行了总结。

袋装分类器的准确性通常显著高于单一分类器的准确性

D ，原始训练数据。它通常对噪声数据和过拟合的影响更具鲁棒性。准确性的提高是因为复合模型降低了各个分类器的方差。

6.7.3 提升

我们现在来看提升的集成方法。与前一节一样，假设作为一名患者，你有某些症状。你选择咨询几位医生，而不是只咨询一位。假设你根据他们之前诊断的准确性，给每位医生的诊断分配权重。最终的诊断是这些加权诊断的组合。这就是提升的本质。

在提升中，每个训练元组也会被分配权重。一系列的 k 分类器是迭代地学习的。在学习完一个分类器 M_i 后，权重会被更新，以便让后续的分类器， M_{i+1} ，旨在“更加关注”被 M_i 错误分类的训练样本。最终的增强分类器， M^* ，结合了每个单独分类器的投票，其中每个分类器投票的权重是其准确性的函数。

AdaBoost（自适应提升的缩写）是一种流行的提升算法。假设我们想要提高学习方法的准确性。我们给定 D ，一个包含 d 类别标记元组的数据集， $(X_1, y_1), (X_2, y_2), \dots, (X_d, y_d)$ ，其中 y_i 是元组 X_i 的类别标签。最初，AdaBoost为每个训练元组分配相等的权重 $1/d$ 。生成 k 个分类器以形成集成需要 k 轮

算法：AdaBoost。一种提升算法——创建一个分类器的集成。每个分类器给出加权投票。

输入：

- D ，一个包含 d 类别标记训练元组的数据集；
- k ，轮数（每轮生成一个分类器）；
- 一个分类学习方案。

输出： 一个复合模型。

方法：

- (1) 将 D 中每个元组的权重初始化为 $1/d$ ；
- (2) 对于 $i=1$ 到 k 执行 // 对于每一轮：
- (3) 样本 D 根据元组权重进行替换抽样以获得 D_i ；
- (4) 使用训练集 D_i 来推导模型， M_i ；
- (5) 计算 $error(M_i)$ ，即 M_i 的错误率（公式 (6.34)）
- (6) 如果 $error(M_i) > 0.5$ 则
- (7) 中止循环；
- (8) 结束如果
- (9) 对于 D 中每个被正确分类的元组 **do**
- (10) 将元组的权重乘以 $error(M_i)/(1-error(M_i))$ ；// 更新权重
- (11) 对每个元组的权重进行归一化。
- (12) **endfor**

要使用集成方法对元组进行分类， X ：

- (1) 将每个类别的权重初始化为 0；
- (2) 对于 $i=1$ 到 k ，执行 // 对于每个分类器：
- (3) $w_i = \log \frac{1-error(M_i)}{error(M_i)}$ // 分类器投票的权重
- (4) $c = M_i(X)$ ；// 从 M_i 获取 X 的类别预测
- (5) 将 w_i 添加到类别 c 的权重中
- (6) **endfor**
- (7) 返回权重最大的类别。

图 6.26

AdaBoost，一种提升算法。

通过算法的其余部分。我们可以抽样形成任何大小的训练集，不一定是大小 d 。使用有放回抽样——同一个元组可能被多次选择。每个元组被选择的概率基于其权重。分类器模型， M_i ，源自 D_i 的训练元组。然后使用 D 作为测试集计算其错误。元组的权重随后根据它们的分类情况进行调整。

如果一个元组被错误分类，它的权重会增加。如果一个元组被正确分类，它的权重会减少。元组的权重反映了分类的难度——权重越高，表示它被错误分类的次数越多。这些权重将用于生成下一轮分类器的训练样本。基本思想是，当我们构建一个分类器时，我们希望它更加关注上一轮的错误分类元组。有些分类器可能在分类某些“困难”元组方面表现得比其他分类器更好。通过这种方式，我们构建了一系列相互补充的分类器。

该算法在图 6.26 中进行了总结。

现在，让我们来看一下算法中涉及的一些数学内容。为了计算模型 M_i 的错误率，我们对 D 中被 M_i 错误分类的每个元组的权重进行求和。也就是说，

$$\text{error}(M_i) = \sum_{j=1}^d w_j \times \text{err}(X_j), \quad (6.34)$$

其中 $\text{err}(X_j)$ 是元组 X_j 的错误分类率：如果元组被错误分类，则 $\text{err}(X_j)$ 为1；否则为0。如果分类器 M_i 的性能如此糟糕，以至于其错误率超过0.5，则我们放弃它。相反，我们通过生成一个新的 D_i 训练集来重新尝试，从中得出一个新的 M_i 。

分类器 M_i 的错误率影响训练元组的权重更新。如果在第 i 轮中某个元组被正确分类，则其权重乘以 $\text{error}(M_i)/(1-\text{error}(M_i))$ 。一旦所有正确分类的元组的权重被更新，所有元组（包括错误分类的元组）的权重都会被归一化，以确保它们的总和保持与之前相同。为了归一化一个权重，我们将其乘以旧权重的总和，再除以新权重的总和。因此，错误分类元组的权重增加，而正确分类元组的权重减少，如前所述。

“一旦提升完成，如何使用分类器的集成来预测元组 X 的类别标签？”与每个分类器被赋予相等投票权的袋装法不同，提升法根据分类器的表现为每个分类器的投票分配权重。分类器的错误率越低，它的准确性就越高，因此，它的投票权重应该越高。分类器 M_i 的投票权重为

$$\log \frac{1 - \text{error}(M_i)}{\text{error}(M_i)}. \quad (6.35)$$

对于每个类别 c ，我们对每个将类别 c 分配给 X 的分类器的权重进行求和。权重总和最高的类别是“赢家”，并作为元组 X 的类别预测返回。

梯度提升是另一种强大的提升技术，可用于分类、回归和排序。如果我们使用树（例如，用于分类的决策树、用于回归的回归树）作为基础模型（即弱学习器），则称为梯度树提升，或梯度提升树。图6.27展示了回归任务的梯度树提升算法。其工作原理如下。

梯度树提升算法从一个简单的回归模型 $F(x)$ （第1行）开始，该模型输出一个常数（即所有训练元组的平均输出）。然后，类似于Adaboost，它在每一轮中尝试找到一个新的基础模型（即弱学习器） $M_t(x)$ （第3行）。新构建的基础模型 $M_t(x)$ 被添加到回归模型 $F(x)$ 中（第8行）。换句话说，复合回归模型 $F(x)$ 由 k 个加性基础模型 $M_t(x)$ 组成（ $t=1, \dots, k$ ）。当我们寻找新的基础模型 $M_t(x)$ 时，所有之前构建的基础模型（即 $M_1(x), \dots, M_{t-1}(x)$ ）保持不变。

为了构建新的基础模型 $M_t(x)$ ，我们首先通过当前的回归模型 $F(x)$ 计算每个训练元组的预测输出 $\hat{y}_i$ （第4行），并计算相对于预测输出 $\hat{y}_i$ 的损失函数的负梯度 r_i （第5行）。然后，我们为训练集 $\{(x_1, r_1), \dots, (x_n, r_n)\}$ 拟合一个回归树模型，其中负梯度 r_i 被视为第 i 个训练元组的目标输出值。由于负梯度 r_i （ $i=1, \dots, n$ ）在不同轮次中变化，我们最终得到不同的基础模型 $M_t(x)$ （ $t=1, \dots, k$ ）。

“但是，我们为什么使用负梯度来构建新的基础模型？”假设损失函数 $L(y_i, F(x_i)) = \frac{1}{2} (y_i - \hat{y}_i)^2$ （回想一下，我们为回归树和最小二乘线性回归模型使用了类似的损失函数）。然后，我们可以证明负梯度

算法：用于回归的梯度树提升。

输入：

- D ，一组 n 训练元组 $\{(x_1, y_1), \dots, (x_n, y_n)\}$ ，其中 x_i 是第 i 个训练元组的属性向量， y_i 是其真实的目标输出值；
- k ，轮数（每轮生成一个基础回归模型）；
- 一个差分损失函数 $\text{Loss} = \sum_{i=1}^n L(y_i, F(x_i))$ 。

输出：一个复合回归模型 $F(x)$ 。

方法：

```

(1) 初始化回归模型  $F(x) = \frac{\sum_{i=1}^n y_i}{n}$ ;
(2) for  $t=1$  to  $k$  do // 构建一个新的弱学习器  $M_t(x)$  每轮:
(3)   for  $i=1$  to  $n$  // 每个训练元组:
(4)     calculate  $\hat{y}_i = F(x_i)$ ; // 当前模型  $F(x)$  的预测值
(5)     计算负梯度  $r_i = -\frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i}$ ;
(6)   endfor
(7)   fit a regression tree model  $M_t(x)$  for the training set  $\{(x_1, r_1), \dots, (x_n, r_n)\}$ ;
(8)   更新复合回归模型  $F(x) \leftarrow F(x) + M_t(x)$ .
(9) endfor
```

图 6.27

用于回归的梯度树提升。

$r_i = y_i - \hat{y}_i$ ，这是当前回归模型 $F(x)$ 预测输出值与实际输出值之间的差异（即残差）。换句话说，负梯度 r_i 显示了当前回归模型 $F(x)$ 的“缺陷”（即预测输出与其实输出值之间的距离）。如果我们使用其他损失函数（例如，稳健回归中的 Huber 损失），负梯度不再等于残差 $y_i - \hat{y}_i$ ，但仍然在预测质量方面提供了一个良好的指标，反映当前回归模型 $F(x)$ 在第 i 个训练元组上的表现。因此，负梯度也被称为 伪残差。通过拟合一个关于负梯度的回归树模型（即当前回归模型的“缺陷”）

$F(x)$ 是新构建的基模型， $M_t(x)$ ，预计将显著改善复合回归模型 $F(x)$ 。

除了图 6.27 中的算法外，还有几种梯度树提升的替代设计选择。例如，类似于 Adaboost，我们可以为每个基模型 $M_t(x)$ 学习一个权重，然后复合回归模型 $F(x)$ 变成 k 个基模型的加权和。在实践中，发现缩小新构建的基模型有助于提高复合模型 $F(x)$ 的泛化性能（即 $F(x) \leftarrow F(x) + \eta M_t(x)$ 在第 8 行，其中 $0 < \eta < 1$ 是缩小常数）。回归树 $M_t(x)$ 的叶节点数量 T 在复合模型 $F(x)$ 的学习性能中起着重要作用。也就是说，如果 T 太小， $F(x)$ 可能会对训练集欠拟合，但如果 T 较大，则可能会对训练集过拟合。通常选择 T 在 4 到 8 之间。在给定的轮次 t 中，我们可以使用整个训练集的子样本来构建基模型 $M_t(x)$ 。

配备这种子采样策略的梯度树提升被称为 随机梯度（树）提升，发现它显著提高了复合模型 $F(x)$ 的准确性。一个高度可扩展的端到端梯度树提升系统称为 **XGBoost**，能够处理十亿规模的训练集。XGBoost 在训练梯度方面进行了多项创新。

树提升，包括为稀疏数据设计的新树构建算法、特征子采样（与随机梯度提升中的训练元组子采样相对），以及高效的缓存感知块结构。XGBoost已被数据科学家成功应用于许多数据挖掘挑战，常常取得顶尖的竞争结果。

“提升与装袋相比如何？”由于提升方法专注于错误分类的元组，它有可能使得生成的复合模型对这些数据过拟合。因此，有时生成的“提升”模型可能不如从相同数据中派生的单一模型准确。

装袋对模型过拟合的敏感性较低。虽然两者在与单一模型相比时都能显著提高准确性，但提升通常能实现更高的准确性。

6.7.4 随机森林

我们现在介绍另一种集成方法，称为随机森林。想象一下，集成中的每个分类器都是一个决策树分类器，因此分类器的集合就是一片“森林”。每个决策树都是通过在每个节点随机选择属性来确定分裂生成的。

更正式地说，每棵树依赖于一个随机向量的值，该向量是独立抽样的，并且所有森林中的树具有相同的分布。在分类过程中，每棵树进行投票，返回最受欢迎的类别。

随机森林可以通过袋装法（第 6.7.2 节）与随机属性选择结合构建。给定一个训练集 D ，包含 d 个元组。生成 k 棵决策树以形成集成的总体过程如下。在每次迭代中， i ($i = 1, 2, \dots, k$)，从 D 中有放回地抽取一个训练集 D_i ，包含 d 个元组。也就是说，每个 D_i 是 D 的自助样本（第 6.6.4 节），因此某些元组可能在 D_i 中出现多次，而其他元组可能被排除。设 F 为用于确定每个节点分裂的属性数量，其中 F 远小于可用属性的数量。为了构建决策树分类器 M_i ，在每个节点随机选择 F 个属性作为该节点分裂的候选属性。使用 CART 方法来生长树。树生长到最大尺寸且不进行修剪。以这种方式形成的随机森林，采用随机输入选择，称为 Forest-RI。

另一种随机森林形式，称为 Forest-RC，使用输入属性的随机线性组合。它不是随机选择属性的子集，而是创建新的属性（或特征），这些属性是现有属性的线性组合。也就是说，通过指定生成一个属性。

L ，待组合的原始属性数量。在给定节点上， L 个属性被随机选择并与在 $[-1, 1]$ 上的均匀随机数的系数相加。 F 线性组合被生成，并在这些组合中进行搜索以找到最佳分割。这种形式的随机森林在可用属性较少时非常有用，以减少个别分类器之间的相关性。

随机森林的准确性与 AdaBoost 相当，但对错误和异常值更具鲁棒性。

只要森林中的树木数量足够大，森林的泛化误差就会收敛。因此，过拟合不太可能成为问题。随机森林的准确性取决于个别分类器的强度以及它们之间的依赖性度量。理想情况下，保持个别分类器的强度而不增加它们的相关性。随机森林对每次分割时选择的属性数量不敏感。通常，最多选择 $\log_2 d + 1$ 个属性。

(一个有趣的经验观察是，使用单个随机输入属性可能会导致良好的准确性，通常高于使用多个属性时的准确性。)因为随机森林考虑了许多

每个分裂的属性较少，它们在非常大的数据库上效率高。它们的速度可能比袋装法或提升法更快。随机森林提供了变量重要性的内部估计。

6.7.5 改善类别不平衡数据的分类准确性

在本节中，我们重新审视类别不平衡问题。特别是，我们研究改善类别不平衡数据的分类准确性的方法。

给定二分类数据，如果主要关注的类别（正类）仅由少数元组表示，而大多数元组表示负类，则数据为类别不平衡。对于多类别不平衡数据，各类别的数据分布差异显著，其中主要关注的类别通常是稀有的。类别不平衡问题与成本敏感学习密切相关，其中每个类别的错误成本并不相等。例如，在医学诊断中，错误地将癌症患者诊断为健康（假阴性）的成本远高于将健康患者误诊为癌症（假阳性）。假阴性错误可能导致生命的损失，因此其成本远高于假阳性错误。其他涉及类别不平衡数据的应用包括欺诈检测、从卫星雷达图像中检测油污以及故障监测。

传统分类算法旨在最小化分类过程中所犯错误的数量。它们假设假阳性和假阴性错误的成本是相等的。由于假设类别的分布是平衡的且错误成本相等，因此它们不适用于类别不平衡的数据。本章前面的部分介绍了应对类别不平衡问题的方法。

尽管准确性度量假设类别的成本是相等的，但可以使用考虑不同类型分类的替代评估指标。例如，第6.6.1节介绍了敏感性（或召回率，真正例率）和特异性（真负例率），这些指标有助于评估分类器在不平衡数据上预测类别标签的能力。讨论的其他相关度量包括F1和F β 。第6.6.6节展示了ROC曲线如何绘制敏感性与1-特异性（即假阳性率）。这样的曲线在研究分类器在类别不平衡数据上的表现时可以提供洞察。

在本节中，我们将探讨提高类别不平衡数据分类准确性的通用方法。这些方法包括（1）过采样，（2）欠采样，（3）阈值移动，以及（4）集成技术。前三种方法不涉及对分类模型构建的任何更改。也就是说，过采样和欠采样改变了训练集中元组的分布；阈值移动影响模型在对新数据进行分类时的决策方式。

集成方法遵循第6.7.2节至第6.7.4节中描述的技术。为了便于解释，我们将这些通用方法描述为针对两类不平衡数据问题，其中高成本类别比低成本类别更为稀少。

过采样和欠采样都会改变训练数据的分布，使得稀有的（正）类别得到良好的表示。过采样通过对正样本进行重采样，使得生成的训练集包含相等数量的正样本和负样本。欠采样通过减少负样本的数量来工作。它随机删除多数（负）类别的样本，直到正样本和负样本的数量相等。

示例 6.12. 过采样和欠采样。 假设原始训练集包含 100 个正例和 1000 个负例。在过采样中，我们复制稀有类别的元组，以形成一个新的训练集，包含 1000 个正例和 1000 个负例。在欠采样中，我们随机

消除负元组，以便新的训练集包含100个正元组和100个负元组。

□

存在几种过采样和欠采样的变体。它们可能在如何添加或消除元组方面有所不同。例如，SMOTE算法使用过采样，其中添加了“接近”给定正元组的合成元组。

阈值移动方法解决类别不平衡问题时不涉及任何采样。它适用于分类器，这些分类器在给定输入元组时返回一个连续的输出值（就像在第6.6.6节中讨论的那样，我们讨论了如何构建ROC曲线）。也就是说，对于输入元组 X ，这样的分类器返回的输出是一个映射， $f(X) \rightarrow [0, 1]$ 。该方法并不通过操纵训练元组，而是基于输出值返回分类决策。在最简单的方法中，对于某个阈值 t ， $f(X) \geq t$ 的元组被视为正类，而所有其他元组被视为负类。其他方法可能涉及通过加权来操纵输出。一般来说，阈值移动会调整阈值 t ，使得稀有类别的元组更容易被分类（因此，出现代价高昂的假阴性错误的机会更小）。此类分类器的例子包括朴素贝叶斯分类器（第6.3节）和神经网络（第10章）。阈值移动方法虽然不如过采样和欠采样流行，但简单且在两类不平衡数据中显示出了一些成功。

集成方法（第6.7.2节至第6.7.4节）也已应用于类别不平衡问题。构成集成的各个分类器可能包括此处描述的方法的版本，例如过采样和阈值移动。

这些方法在二分类任务中的类别不平衡问题上效果相对较好。阈值移动和集成方法在实证上被观察到优于过采样和欠采样。阈值移动在极度不平衡的数据集上也表现良好。多分类任务中的类别不平衡问题要困难得多，过采样和阈值移动的效果较差。尽管阈值移动和集成方法显示出希望，但找到多分类不平衡问题的解决方案仍然是未来的研究方向。

6.8 摘要

- 分类 是一种数据分析形式，提取描述数据类别的模型。分类器或分类模型预测分类标签（类别）。数值预测 模型是连续值函数。分类和数值预测是两种主要的预测问题类型。
- 决策树生成 是一种自顶向下的递归树生成算法，它使用属性选择度量来选择每个非叶节点测试的属性。ID3、C4.5 和 CART 是使用不同属性选择度量的此类算法的示例。树剪枝算法试图通过去除反映数据噪声的树枝来提高准确性。
- 朴素贝叶斯分类 基于贝叶斯定理的后验概率。它假设类条件独立——即属性值对给定类别的影响与其他属性的值是独立的。
- 线性分类器 计算输入属性值的线性加权组合，并基于此预测给定元组的类别标签。感知机和 逻辑回归 是线性分类器的两个经典示例。

- 决策树分类器、贝叶斯分类器和线性分类器都是渴望学习者的例子，因为它们使用训练元组来构建泛化模型，从而准备好对新元组进行分类。这与懒惰学习者或基于实例的分类方法形成对比，例如最近邻分类器和基于案例的推理分类器，它们将所有训练元组存储在模式空间中，并在呈现测试元组之前等待进行泛化。因此，懒惰学习者需要高效的索引技术。
- 混淆矩阵可用于评估分类器的质量。对于二分类问题，它显示真阳性、真阴性、假阳性和假阴性。评估分类器预测能力的指标包括准确率、灵敏度（也称为召回率）、特异性、精确率、F 和 F_{β} 。当主要关注的类别处于少数时，依赖于准确率指标可能会产生误导。
- 构建和评估分类器需要将标记数据划分为训练集和测试集。保留法、随机抽样、交叉验证和自助法用于这种划分的典型方法。
- 显著性检验和ROC曲线是模型选择的有用工具。显著性检验可以用来评估两个分类器之间的准确性差异是否由于偶然因素造成。ROC曲线绘制了一个或多个分类器的真正例率（或灵敏度）与假正例率（或1-特异性）之间的关系。
- 集成方法可以通过学习和组合一系列个体（基础）分类器模型来提高整体准确性。装袋法、提升法和随机森林是流行的集成方法。
- 类别不平衡问题发生在主要关注的类别仅由少数元组表示时。解决此问题的策略包括过采样、欠采样、阈值移动和集成技术。

6.9 练习

- 6.1. 简要概述决策树分类的主要步骤。
- 6.2. 为什么树剪枝在决策树归纳中是有用的？使用单独的元组集来评估剪枝的缺点是什么？
- 6.3. 给定一个决策树，您可以选择 (a) 将决策树转换为规则，然后修剪生成的规则，或 (b) 修剪决策树，然后将修剪后的树转换为规则。(a) 相对于 (b) 有什么优势？
- 6.4. 计算决策树算法的最坏情况计算复杂度是很重要的。给定数据集 D ，属性数量 n ，以及训练元组数量 $|D|$ ，分析计算复杂度与 n 和 $|D|$ 的关系。
- 6.5. 给定一个 5-GB 的数据集，包含 50 个属性（每个属性包含 100 个不同的值）和 512 MB 的主内存，概述一种在如此大数据集中构建决策树的高效方法。通过对主内存使用情况的粗略计算来证明您的答案。
- 6.6. 为什么朴素贝叶斯分类被称为“朴素”？简要概述朴素贝叶斯分类的主要思想。
- 6.7. 以下表格包含来自员工数据库的训练数据。数据已被概括。例如，“31... 35”表示年龄的范围为 31 到 35。对于给定的

行条目，*count* 表示具有该行中给定的 部门、状态、年龄 和 薪水 值的数据元组数量。

部门	状态	年龄	薪资	数量
销售	高级	31...35	46K...50K	30
销售	初级	26...30	26K...30K	40
销售	初级	31...35	31K...35K	40
系统	初级	21...25	46K...50K	20
系统	高级	31...35	66K...70K	5
系统	初级	26...30	46K...50K	3
系统	高级	41...45	66K...70K	3
市场营销	高级	36...40	46K...50K	10
市场营销	初级	31...35	41K...45K	4
秘书	高级	46...50	36K...40K	4
秘书	初级	26...30	26K...30K	6

让 状态 成为类别标签属性。

- a. 您将如何修改基本的决策树算法，以考虑每个通用数据元组（即每行条目）的计数？
 - b. 使用您的算法从给定数据构建决策树。
 - c. 给定一个数据元组，其属性值为“系统”，“26...30”，和“46-50K”
部门、年龄、和薪水，分别，天真的贝叶斯分类将会是什么
元组的状态是什么？
- 6.8. 比较 *eager* 分类的优缺点（例如，决策树，贝叶斯，神经网络）与 *懒惰* 分类（例如，*k*-最近邻，基于案例的推理）。
- 6.9. 编写一个算法用于 *k*-最近邻分类，给定 *k*，最近的邻居数量，以及 *n*，描述每个元组的属性数量。
- 6.10. RainForest 是一个可扩展的决策树归纳算法。开发一个可扩展的朴素贝叶斯分类算法，该算法在大多数数据库中只需对整个数据集进行一次扫描。
讨论这样的算法是否可以被改进以结合 提升 进一步提高其分类准确性。
- 6.11. 设计一种高效的方法，在无限的数据流上执行有效的朴素贝叶斯分类（即，您只能扫描数据流一次）。如果我们想要发现这种分类方案的演变（例如，将此时的分类方案与一周前的早期方案进行比较），您会建议什么修改设计？
- 6.12. 感知器模型 $y = f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$ 可用于从训练数据中学习二元分类器。
- a. 假设有两个训练样本。正样本为 $\mathbf{x}_1 = (2, 1)^T$ ；负样本为 $\mathbf{x}_2 = (1, 0)^T$ 。学习率 $\eta = 1$ 。从 $\mathbf{w} = (1, 1)^T$ 和 $b = 0$ 开始，求解分类器的参数。
 - b. 假设有四个训练样本。正样本为 $\mathbf{x}_1 = (1, 1)^T$ 和 $\mathbf{x}_2 = (0, 0)^T$ ；负样本为 $\mathbf{x}_3 = (1, 0)^T$ 和 $\mathbf{x}_4 = (0, 1)^T$ 。我们能否使用感知器模型正确分类所有训练样本？为什么？
- 6.13. 假设我们有三个正例 $\mathbf{x}_1 = (1, 0, 0)$, $\mathbf{x}_2 = (0, 0, 1)$ 和 $\mathbf{x}_3 = (0, 1, 0)$ ，以及三个负例 $\mathbf{x}_4 = (-1, 0, 0)$, $\mathbf{x}_5 = (0, -1, 0)$ 和 $\mathbf{x}_6 = (0, 0, -1)$ 。应用标准的梯度上升法训练一个逻辑回归分类器（不带正则化项）。

元组 #类概率		
1	P	0.95
2	N	0.85
3	P	0.78
4	P	0.66
5	N	0.60
6	P	0.55
7	N	0.53
8	N	0.52
9	N	0.51
10	P	0.40

图 6.28

按得分降序排列的元组，其中得分是概率分类器返回的值。

用两个不同的值初始化权重向量，并设置 $w^0 = 0$ （例如 $w_0 = (0, 0, 0, 0)'$ ， $w_0 = (0, 0, 1, 0)'$ ）。对于这两个不同的初始值，最终的权重向量 (w^*) 会是相同的吗？值是什么？请详细解释你的答案。你可以假设学习率为一个正实常数 η 。

6.14. 假设我们正在训练一个朴素贝叶斯分类器和一个逻辑回归分类器： $f: \mathbf{X} \rightarrow Y$ ，它将一个 d 维实值特征向量 $\mathbf{X} \in \mathbb{R}^d$ 映射到一个二元类标签 $Y \in \{0, 1\}$ 。在朴素贝叶斯分类器中，我们假设所有的 $\mathbf{X}_i$ （其中 $i=1, \dots, n$ ）在给定类标签 Y 和类先验 $P(Y)$ 的情况下是条件独立的，并且类标签 Y 服从伯努利分布，且 $P(Y=1)=\theta$ 。现在，证明在这两个假设下逻辑回归和朴素贝叶斯的等价性。

- 对于每个 $\mathbf{X}_i$ ，我们假设它是从高斯分布 $P(\mathbf{X}_i|Y=k) \sim \mathcal{N}(\mu_{ik}, \sigma_{ik})$ 中抽取的，其中 $k=0, 1$ 。我们还假设 $\sigma_{i0}=\sigma_{i1}=\sigma_i$ 。
- 对于每个 $\mathbf{X}_i$ ，我们假设它是从伯努利分布中抽取的 $P(\mathbf{X}_i=1|Y=k)=p_k$ 其中 $k=0, 1$ 。

6.15. 证明准确性是敏感性和特异性的函数，即证明公式 (6.25)。

6.16. 调和平均数是几种平均数中的一种。第2章讨论了如何计算算术平均数，这是大多数人在计算平均数时通常想到的。

正实数 $x_1, x_2, \dots, x_n$ 的调和平均数 H 定义为

$$H = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_n}}$$

$$= \frac{n}{\sum_{i=1}^n \frac{1}{x_i}}.$$

F 评分是精确率和召回率的调和平均数。利用这一事实推导公式 (6.28) 关于 F 的表达。此外，将 F_β 写成真正正例、假负例和假正例的函数。

6.17. 图6.28的数据元组按分类器返回的概率值降序排列。

对于每个元组，计算真正正例(TP)、假正例(FP)、真正负例(TN)和假负例(FN)的值。计算真正正例率(TPR)和假正例率(FPR)。绘制数据的ROC曲线。

- 6.18. 当单个数据对象可能同时属于多个类别时，评估分类 *准确性* 是困难的。在这种情况下，评论一下您将使用什么标准来比较基于相同数据建模的不同分类器。
- 6.19. 假设我们想在两个预测模型 M_1 和 M_2 之间进行选择。我们对每个模型进行了10轮10折交叉验证，其中第 i 轮使用相同的数据划分，适用于 M_1 和 M_2 。对于 M_1 获得的错误率为30.5、32.2、20.7、20.6、31.0、41.0、27.7、26.0、21.5、26.0。对于 M_2 的错误率为22.4、14.5、22.4、19.6、20.7、20.4、22.1、19.4、16.2、35.0。请评论在1%的显著性水平下，是否有一个模型显著优于另一个模型。
- 6.20. 什么是 *boosting*？说明它为什么可能提高决策树归纳的准确性。
- 6.21. 概述解决 *类别不平衡问题* 的方法。假设一家银行想要开发一个分类器，以防止欺诈性信用卡交易。说明如何基于大量合法示例和非常少量的欺诈案例来诱导一个高质量的分类器。
- 6.22. XGBoost是一个可扩展的树提升机器学习系统。它的目标函数包含一个训练损失和一个正则化项： $\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k (f_k)$ 。阅读XGBoost论文并回答以下问题：
- 什么是 $\hat{y}_i$ ？在第 t 次迭代中，XGBoost 固定 $f_1, \dots, f_{t-1}$ ，并训练第 t 棵树模型 f_t 。XGBoost 如何近似训练损失 $l(y_i, \hat{y}_i)$ ？
 - 什么是 $\Omega(f_k)$ ？在第 t 次

迭代中，正则化项中需要考虑哪一部分？

6.10 参考文献笔记

分类是机器学习、统计学和模式识别中的一个基本主题。许多来自这些领域的教科书强调了分类方法，例如 Mitchell [Mit97]；Bishop [Bis06a]；Duda、Hart 和 Stork [DHS01]；Theodoridis 和 Koutroumbas [TK08]；Hastie、Tibshirani 和 Friedman [HTF09]；Alpaydin [Alp11]；Marsland [Mar09]；以及 Aggarwal [Agg15a]。

关于决策树归纳，C4.5 算法在 Quinlan [Qui93] 的书中有描述。CART 系统在 Breiman、Friedman、Olshen 和 Stone [BFOS84] 的《分类与回归树》中详细介绍。这两本书对决策树归纳的许多问题进行了出色的阐述。C4.5 有一个商业继任者，称为 C5.0，可以在 <http://www.rule-quest.com> 找到。ID3，C4.5 的前身，在 Quinlan [Qui86] 中有详细描述。它扩展了 Hunt、Marin 和 Stone [HMS66] 对概念学习系统的开创性工作。

其他决策树生成算法包括 FACT (Loh 和 Vanichsetakul [LV88])，QUEST (Loh 和 Shih [LS97])，PUBLIC (Rastogi 和 Shim [RS98])，以及 CHAID (Kass [Kas80] 和 Magidson [Mag94])。INFERULE (Uthurusamy, Fayyad 和 Spangler [UFS91]) 从不确定数据中学习决策树，获得的是概率而非分类的分类规则。

KATE (Manago 和 Kodratoff [MK91]) 从复杂结构数据中学习决策树。ID3 的增量版本包括 ID4 (Schlimmer 和 Fisher [SF86]) 和 ID5 (Utgoff [Utg88])，后者在 Utgoff、Berkman 和 Clouse [UBC97] 中进行了扩展。CART 的增量版本在 Crawford [Cra89] 中描述。BOAT (Gehrke、Ganti、Ramakrishnan 和 Loh [GGRL99]) 是一种解决数据挖掘中可扩展性问题的决策树算法，也具有增量特性。其他决策树

解决可扩展性的算法包括SLIQ (Mehta、Agrawal和Rissanen [MAR96])、SPRINT (Shafer、Agrawal和Mehta [SAM96])、RainForest (Gehrke、Ramakrishnan和Ganti [GRG98]) 以及早期的方法,如Catlet [Cat91]和Chan与Stolfo [CS93a,CS93b]。

有关决策树归纳的许多显著问题的综合调查,例如属性选择和剪枝,请参见Murthy [Mur98]。基于感知的分类(PBC),一种视觉和交互式的决策树构建方法,见于Ankerst、Elsen、Ester和Kriegel [AEEK99]。

有关属性选择度量的详细讨论,请参见Kononenko和Hong [KH97]。信息增益由Quinlan [Qui86]提出,基于Shannon和Weaver [SW49]在信息理论方面的开创性工作。增益比作为信息增益的扩展被描述为C4.5的一部分(Quinlan [Qui93])。Gini不纯度是由Breiman、Friedman、Olshen和Stone [BFOS84]为CART提出的。基于信息理论的G统计量在Sokal和Rohlf [SR81]中给出。属性选择度量的比较包括Buntine和Niblett [BN92]、Fayyad和Irani [FI92]、Kononenko [Kon95]、Loh和Shih [LS97]以及Shih [Shi99]。Fayyad和Irani [FI92]显示了基于不纯度的度量(如信息增益和Gini不纯度)的局限性。他们提出了一类称为C-SEP(类分离)的属性选择度量,在某些情况下优于基于不纯度的度量。

Kononenko [Kon95]指出,基于最小描述长度原则的属性选择度量对多值属性的偏差最小。Martin和Hirschberg [MH95]证明,在最坏情况下,决策树归纳的时间复杂度随着树的高度呈指数级增长,而在相当一般的条件下,平均情况下也是如此。Fayyad和Irani [FI90]发现,浅层决策树往往具有许多叶子节点,并且在各种领域中错误率较高。属性(或特征)构造在Liu和Motoda [LM98,Le98]中进行了描述。

决策树剪枝有许多算法,包括成本复杂度剪枝(Breiman、Friedman、Olshen和Stone [BFOS84])、减少错误剪枝(Quinlan [Qui87])和悲观剪枝(Quinlan [Qui86])。PUBLIC (Rastogi和Shim [RS98])将决策树构建与树剪枝相结合。基于MDL的剪枝方法可以在Quinlan和Rivest [QR89]、Mehta、Agrawal和Rissanen [MAR96]以及Rastogi和Shim [RS98]中找到。其他方法包括Niblett和Bratko [NB86]以及Hosking、Pednault和Sudan [HPS97]。有关剪枝方法的实证比较,请参见Mingers [Min89]和Malerba、Floriani和Semeraro [MFS95]。有关简化决策树的调查,请参见Breslow和Aha [BA97]。

贝叶斯分类的详细介绍可以在Duda、Hart和Stork [DHS01]、Weiss和Kulikowski [WK91]以及Mitchell [Mit97]中找到。关于当类条件独立假设被违反时,朴素贝叶斯分类器的预测能力分析,请参见Domingos和Pazzani [DP96]。在John [Joh97]中,报告了对连续值属性进行核密度估计而非高斯估计的朴素贝叶斯分类器的实验。

最近邻分类器由Fix和Hodges [FH51]于1951年提出。关于最近邻分类的综合文章集可以在Dasarathy [Das91]中找到。更多参考文献可以在许多分类相关的书籍中找到,例如Duda、Hart和Stork [DHS01]以及James [Jam85],还有Cover和Hart [CH67]以及Fukunaga和Hummels [FH87]的文章。

它们与属性加权 and 噪声实例的剪枝的结合在Aha [Aha92]中进行了描述。使用搜索树来提高最近邻分类时间的详细信息见Friedman、Bentley和Finkel [FBF77]。部分距离方法是由向量量化和压缩领域的研究人员提出的,概述见Gersho和Gray [GG92]。用于重新编辑的方法-

将“无用”的训练元组移动的想法最早由哈特（Hart）提出[Har68]。有关基于局部敏感哈希的k近邻计算加速，请参见潘（Pan）和马诺查（Manocha）[PM12,PM11]；以及张（Zhang）、黄（Huang）、耿（Geng）和刘（Liu）[ZHGL13]。

最近邻分类器的计算复杂性在Preparata和Shamos的著作中进行了描述[PS85]；以及Hagani、Sebastian和Karl[HMA09]。关于基于案例的推理的参考文献包括Riesbeck和Schank的著作[RS89]和Kolodner[Kol93]，以及Leake[Lea96]和Aamodt与Plazas[AP94]。有关商业应用的列表，请参见Allen[Ail94]。医学中的例子包括Koton的CASEY[Kot88]和Bareiss、Porter与Weir的PROTOS[BPW88]，而Rissland和Ashley[RA87]则是法律领域的CBR示例。CBR在多个商业软件产品中可用。

线性回归及其众多变体，如岭回归（RIDGE regression）、稳健回归（robust regression），在大多数统计学教科书中都有涉及，如Freedman[Fre09]；Draper和Smith[DS98]；以及Fox[Fox97]。有关LASSO，请参见Tibshirani[Tib11]。感知器（Perceptron）最早由罗森布拉特（Rosenblatt）发明[Ros58]，诺维科夫（Novikoff）分析了其收敛性[Nov63]。感知器是最早的线性分类器之一，由罗森布拉特于1958年提出[Ros58]，成为早期机器学习历史上的一个里程碑。1969年，明斯基（Minsky）和帕珀特（Papert）[MP69]证明感知器无法学习线性不可分的概念。有关逻辑回归的一般介绍可以在大多数机器学习教科书中找到，如Mitchell[Mit97]；Hastie、Tibshirani和Friedman[HTF09]；以及Aggarwal[Agg15a]。Ng和Jordan[NJ02]对朴素贝叶斯分类器和逻辑回归进行了全面比较。逻辑回归与对数线性模型之间的关系可以在Christensen[Chr06]中找到。

在Weiss和Kulikowski [WK91]以及Witten和Frank [WF05]中描述了估计分类器准确性所涉及的问题。大多数信息检索教科书中讨论了敏感性、特异性和精确度。有关F和F β 度量，请参见van Rijsbergen [vR90]。基于Kohavi [Koh95]的理论和实证研究，推荐使用分层10折交叉验证来估计分类器的准确性，优于保留法、交叉验证、留一法（Stone [Sto74]）和自助法（Efron和Tibshirani [ET93]）。有关置信区间和显著性统计检验，请参见Freedman、Pisani和Purves [FPP07]。

有关ROC分析，请参见Egan [Ega75]、Swets [Swe88]以及Vuk和Curk [VC06]。Breiman [Bre96]提出了Bagging。Freund和Schapire [FS97]提出了AdaBoost。这种提升技术已应用于几种不同的分类器，包括决策树归纳（Quinlan [Qui96]）和朴素贝叶斯分类（Elkan [Elk97]）。Friedman [Fri01]提出了梯度提升机。Chen和Guestrin设计了一个高度可扩展的系统，称为Xgboost [CG16]。随机森林的集成技术由Breiman [Bre01]描述。Seni和Elder [SE10]提出了重要性采样学习集成（ISLE）框架，该框架将Bagging、AdaBoost、随机森林和梯度提升视为通用集成生成过程的特例。

有许多在线软件包用于集成例程，包括装袋法、AdaBoost、梯度提升和随机森林。关于类别不平衡问题和/或成本敏感学习的研究包括Weiss [Wei04]、Zhou和Liu [ZL06]、Zapkowicz和Stephen [ZS02]、Elkan [Elk01]、Domingos [Dom99]，以及Huang、Li、Loy和Tang [HLLT16]。

加利福尼亚大学尔湾分校（UCI）维护一个机器学习数据集库，用于分类算法的开发和测试。它还维护一个知识发现数据库（KDD）档案，这是一个在线的大型数据集库，涵盖了各种数据类型、分析任务和应用领域。

有关这两个数据集库的信息，请参见

<http://www.ics.uci.edu/~mllearn/MLRepository.html> 和 <http://kdd.ics.uci.edu>。

没有任何分类方法在所有数据类型和领域中都优于其他方法。分类方法的实证比较包括 Quinlan [Qui88]; Shavlik、Mooney 和 Towell [SMT91]; Brown、Corruble 和 Pittard [BCP93]; Curram 和 Mingers [CM94]; Michie、Spiegelhalter 和 Taylor [MST94]; Brodley 和 Utgoff [BU95]; 以及 Lim、Loh 和 Shih [LLS00]。

